

Full length article

STAC-IoT: A secure task-based access control for IoT-edge computing architecture

Hicham Degdeg^a,^{*}, Mohamed Lehsaini^a, Youcef Imine^b^a STIC Laboratory, Computer Science Department, University of Tlemcen, Tlemcen, Algeria^b LAMIH, CNRS, UMR 8201, Université Polytechnique Hauts-de-France, 59313 Valenciennes Cedex 9, France

ARTICLE INFO

Keywords:

IoT security
Access control
Edge computing
Attribute-based encryption
Secret sharing
Multi-authority

ABSTRACT

The explosive growth of the Internet of Things (IoT) has revolutionized many sectors by enabling smart devices to communicate and interact autonomously. Decentralized architectures, such as edge computing, have emerged to enable a seamless management of IoT. However, this technological coupling brings significant security challenges, especially when controlling access to tasks executed by IoT objects. In fact, most existing solutions rely on a centralized trusted node, lack fine-grained access control, and are not adapted to resource-constrained environments. In this paper, we propose STAC-IoT, a new decentralized task-based access control protocol that governs access to actions executed by IoT objects. The proposed approach leverages attribute-based access control model and edge computing to enable a robust, secure and energy efficient task execution. Furthermore, the proposed protocol supports the existence of multiple authorities that independently manage their own users, while being able to adopt cooperative access policies. The treatment of access requests are fast as our protocol eliminates the need to go through a central authority for every access attempt. To demonstrate the applicability of our protocol in a real-world context, we validate it through a realistic smart manufacturing scenario, demonstrating its effectiveness in terms of scalability, security, and energy efficiency. In addition, we provide a formal security analysis showing that STAC-IoT is resilient against common attack vectors such as man-in-the-middle, impersonation, and token forgery.

1. Introduction

In recent years, the proliferation of connected devices has led to the rapid expansion of the IoT (Bauer et al., 2013), enabling new forms of automation and data-driven decision-making across various domains (Sundmaeker et al., 2010; Gubbi et al., 2013). This massive growth has introduced new challenges related to scalability, responsiveness, and efficient processing of large volumes of data. To address the computational and connectivity limitations inherent in the IoT, edge computing has emerged as a key architectural paradigm. By decentralizing processing and storage closer to the data sources, it reduces latency, alleviates the load on centralized infrastructure, and improves responsiveness, especially in time-sensitive and autonomous environments (Shi et al., 2016).

Despite these advantages, securing edge-enabled IoT environments remains a major challenge. Among the key security requirements, access control plays a central role in ensuring that only authorized entities can interact with sensitive resources or trigger critical operations (Ravidas et al., 2019). However, traditional access control mechanisms are often inadequate for IoT and Edge environments, as

they typically rely on centralized architectures and demand significant computational resources, which are incompatible with the distributed and resource-constrained nature of these environments (Ouaddah et al., 2017).

A broad spectrum of access control mechanisms has been proposed in the literature to address the specific constraints of IoT and edge-enabled systems. Notably, schemes such as (Li et al., 2019; Zhang et al., 2025; Aghili et al., 2022; Sarma et al., 2022; Zhang et al., 2022; Xiang et al., 2022; Yao et al., 2024; Makhdoom et al., 2020; Das and Namasudra, 2022; Li et al., 2022) primarily focus on securing access to the data. In scenarios such as remote actuation and industrial process control, granting access to specific device actions requires a more expressive and specialized access control framework. This level of control cannot be achieved using the mechanisms described above. To bridge this gap, a number of studies have investigated action-based access control mechanisms (Zhang et al., 2021; Fedrecheski et al., 2018; Ameer et al., 2022; Beltrán, 2018; Oh et al., 2019; Pal et al., 2019; Ameer et al., 2020, 2023; Sivaselvan et al., 2023; Khalid et al., 2023). While these contributions represent significant progress, the vast

^{*} Corresponding author.E-mail addresses: hicham.degdeg@univ-tlemcen.dz (H. Degdeg), mohamed.lehsaini@univ-tlemcen.dz (M. Lehsaini), youcef.imine@uphf.fr (Y. Imine).

majority rely on single-authority architectures. Such designs inherently suffer from limited scalability, a lack of interoperability across domains, and an increased risk of a single point of failure (Trabelsi et al., 2023). Moreover, these schemes are often not well suited to modern IoT ecosystems, as they lack support for cross-domain access control. As a result, they are unable to handle scenarios where users belong to multiple organizations and access policies depend on attributes managed by different authorities.

To address the limitations of centralized access control, some studies have explored the use of blockchain technologies (Zaidi et al., 2023; Wang et al., 2023; Ali et al., 2019; Sun et al., 2021; Liu et al., 2020). These approaches typically rely on smart contracts to enforce access policies and support decentralized management through permissioned blockchain infrastructures. While they enhance auditability and system resilience, their integration into IoT ecosystems introduces significant challenges (Sun et al., 2025).

As highlighted by Al Sadawi et al. (2021), deploying blockchain within IoT environments requires advanced technical infrastructures and leads to considerable computational and operational overhead. In particular, blockchain-based access control mechanisms often incur high latency and gas fees, limiting their practicality in large-scale and resource-constrained environments. In addition, many of these models lack the fine-grained flexibility provided by attribute-based access control, thereby reducing their adaptability to dynamic and heterogeneous IoT contexts.

Despite these advancements, a critical gap persists in the literature: no existing solution simultaneously addresses the threefold requirement of (i) fine-grained and task-based access control, (ii) decentralized enforcement with multi-authority support, and (iii) lightweight, low-latency operation suitable for resource-constrained IoT environments.

Motivated by these limitations, this paper proposes a novel access control protocol specifically designed for decentralized and resource-constrained IoT environments. Our approach enables collaborative policy management among multiple authorities, ensures fine-grained access control over actions, and minimizes computational and communication costs through the integration of lightweight cryptographic protocols.

The proposed scheme allows IoT devices to independently verify access requests without relying on centralized authorities or third-party auditors. This design eliminates single points of failure and reduces communication delays. Moreover, users do not need to repeatedly contact a central node to obtain access rights, which significantly reduces system overhead. The scheme is specifically designed to maintain strong security guarantees while remaining lightweight and scalable, making it well suited for real-world deployment in distributed IoT environments.

Our main contributions are summarized as follows:

- (1) We propose STAC-IoT, a novel decentralized access control protocol tailored for IoT environments, enabling multiple authorities to collaboratively enforce access policies using Multi-Authority Attribute-Based Encryption (MA-ABE).
- (2) We design a scalable edge-based architecture in which edge nodes manage groups of IoT devices and securely distribute the necessary credentials and policies. This allows access requests to be processed and enforced locally by the devices, reducing communication overhead and removing dependence on central servers.
- (3) We provide a formal security analysis, demonstrating that the proposed scheme is resilient against common attack vectors, including man-in-the-middle attacks, impersonation attacks, and token forgery.
- (4) We evaluate the effectiveness and practicality of our protocol through both a realistic smart manufacturing simulation and an experimental performance analysis, highlighting its suitability for resource-constrained and latency-sensitive IoT environments.

The remainder of this paper is organized as follows: related work is reviewed in Section 2. Section 3 provides the necessary background on the mechanisms used in our solution. The proposed approach is presented in Section 4. In Section 5, we conduct a detailed security analysis. Section 6 introduces a practical use case and evaluates the performance of our scheme. A broader discussion is provided in Section 7. Finally, Section 8 concludes the paper.

2. Related work

Since the emergence of IoT, many access control solutions have been developed to manage interactions with connected devices. Given the heterogeneous nature of IoT environments and the limited computational and energy resources of smart objects, designing access control mechanisms that ensure robust security, scalability, and lightweight execution remains a major research challenge. In this section, we classify existing access control approaches according to their methodological foundations and identify their main limitations, particularly in the context of collaborative and decentralized IoT ecosystems.

Li et al. (2019) introduced a powerful cryptographic primitive, namely the CCA2 Secure Publicly-Verifiable Revocable Large-Universe Multi-Authority Attribute-Based Encryption (CCA2-PV-R-LU-MA-ABE). This scheme enables fine-grained access control across multiple domains in fog computing, while supporting revocation and public verifiability. Although the model supports multi-authority collaboration, it is exclusively designed for data-level access control, offering no support for managing task execution on IoT devices. In a similar direction, Sarma et al. (2022) proposed a multi-authority CP-ABE scheme targeting secure data access in fog-based environments, called MACFI. Despite its decentralized design, MACFI like the previous scheme is limited to data-level policies, without extending to action-level authorizations. Zhang et al. (2022) proposed a lightweight decentralized multi-authority access control scheme based on CP-ABE and blockchain, but the scheme remains focused on data protection. Similarly, Xiang et al. (2022) introduced a blockchain-based access control protocol tailored to emergency healthcare systems (EHS), which emphasizes secure data sharing but does not address the control of device-level operations.

In contrast, more recent efforts have started addressing task-level access control, which is critical for IoT use cases involving actuation, command execution, or service delegation. Firstly, Ameer et al. (2022) introduced an attribute-based access control model tailored to smart home environments, called HABAC, which supports context-aware, task-based authorization. While this solution represents a step forward toward action-oriented control, it still relies on a centralized authority for managing IoT devices and is primarily designed for small-scale environments, such as smart homes. This significantly limits its applicability in industrial or large-scale IoT environments. Sivaselvan et al. (2023) proposed a blockchain-based scheme that employs capability tokens as transferable rights to perform specific operations on IoT devices. However, their design depends on interface certification for registration and for handling communication between IoT nodes and the blockchain network. This introduces challenges related to certificate management, especially in large-scale deployments. Fedrecheski et al. (2018) presented a distributed ABAC system for Swarm environments, relying on a local broker integrating “Policy Decision Point” (PDP), “Policy Information Point” (PIP), and “Policy Administration Point” (PAP) components. Nevertheless, their approach is built on a centralized client-server model where the “Policy Enforcement Point” (PEP) acts as the client and PDP as the server. This architecture suffers from two major weaknesses: (i) a single point of failure, and (ii) communication over unencrypted and unauthenticated HTTP channels, which significantly increases security risks. Huang et al. (2023) propose ZT-Access, a zero trust access control scheme that combines CP-ABE with real-time trust evaluation to mitigate risks from compromised devices in power IoT environments. The system enforces access by jointly validating attribute policies and dynamic trust scores computed

from behavioral data. While ZT-Access enhances security by adapting to malicious behavior and preserving attribute privacy, it relies on centralized components such as a Trust Management Center and a single Attribute Authority, which may introduce bottlenecks and single points of failure. In addition, the approach incurs significant computational complexity due to frequent trust re-evaluations and requires continuous monitoring infrastructures, which may limit its scalability and applicability in lightweight or large-scale IoT deployments.

Although these solutions attempt to address action-level access, many of them still rely on centralized control components for user authentication and policy management. This centralization limits scalability and resilience, particularly in multi-domain IoT ecosystems where resources and users span various administrative boundaries. To overcome these limitations, a number of studies have explored blockchain-based access control, aiming to provide decentralized and tamper-proof enforcement. Hao et al. (2022) proposed a cross-domain access control model based on a consortium blockchain and Role-Based Access Control (RBAC). In this approach, user roles define access rights, and each request is validated by the blockchain network before a token is generated and stored. While this design improves resistance to attacks and enables interoperability, it lacks fine-grained access control and does not support multi-authority configurations. Moreover, the reliance on PBFT consensus limits the system's scalability, and the use of paid blockchain transactions introduces operational costs. Zaidi et al. (2023) combine RBAC with blockchain by implementing smart contracts on Hyperledger Fabric to control access to IoT device actions. While this hybrid model achieves decentralization, it remains vulnerable to role explosion in dynamic environments and lacks the expressiveness of ABAC models. Wang et al. (2023) present a decentralized access control framework for Industrial IoT (IIoT) based on identity tokens and smart contracts deployed on Ethereum. The system architecture includes full nodes (local servers managing IIoT devices), light nodes (user-side clients), and a set of smart contracts that handle token issuance and user registration. NTRU encryption is used to ensure lightweight privacy-preserving communication. Although the approach enables decentralized and traceable access management, it lacks support for fine-grained, attribute-based policies and does not incorporate multi-authority governance, limiting its flexibility in dynamic and cross-domain IoT settings. Liu et al. (2020) propose Fabric-IoT, a blockchain-based access control system that integrates Attribute-Based Access Control (ABAC) with Hyperledger Fabric. The architecture includes smart contracts for device data (DC), policy management (PC), and access control (AC), enabling decentralized, fine-grained, and dynamic access decisions. A smart gateway bridges resource-constrained IoT devices with the blockchain by offloading data storage and authorization requests. While the system supports policy automation and traceability, it assumes centralized admin roles and does not address cross-domain or multi-authority scenarios.

Despite their advantages in decentralization and auditability, blockchain-based schemes face several technical challenges, such as computational complexity, transaction latency, and infrastructure overhead. These limitations make them impractical for large-scale and resource-constrained IoT environments.

In summary, although a broad range of access control models have been proposed, existing solutions still face fundamental trade-offs between decentralization, fine-grained expressiveness, task-level enforcement, and system efficiency as shown in Table 1. There remains a pressing need for a lightweight, scalable, and collaborative access control protocol capable of managing task execution in decentralized, heterogeneous, and large-scale IoT environments.

3. Background

3.1. Bilinear map and bilinear groups

Let G_1 be cyclic group of prime order p , and let G_T be a target group. A bilinear map is a function $e : G_1 \times G_1 \rightarrow G_T$ that satisfies the following properties:

Table 1

Comparison of related access control schemes for IoT environments.

Scheme	Access to tasks	Decentralization	Multi-authority	Fine-grained access
Li et al. (2019)	✗	✓	✓	✓
Sarma et al. (2022)	✗	✗	✓	✓
Fedrecheski et al. (2018)	✓	✓	✓	✗
Sivaselvan et al. (2023)	✓	✓	✗	✗
Zaidi et al. (2023)	✓	✓	✓	✗
Wang et al. (2023)	✓	✓	✗	✗
Our solution	✓	✓	✓	✓

- (1) **Bilinearity:** for all $g_1, g_2 \in G_1$, and $a, b \in \mathbb{Z}_p$:

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}.$$

- (2) **Non-degeneracy:** there exist elements $g_1, g_2 \in G_1$ such that $e(g_1, g_2) \neq 1$.

- (3) **Efficient Computability:** the function e is efficiently computable in polynomial time.

3.2. Multi-authority attribute-based encryption

The following outlines the main steps of a decentralized attribute-based encryption scheme MA-ABE (Lewko and Waters, 2011).

3.2.1. Global setup

In the global setup, the authorities agree on a bilinear group $\mathbb{G}$ of prime order p , along with a generator $g_1 \in \mathbb{G}$. They also define a cryptographic hash function $H : \{0, 1\}^* \rightarrow \mathbb{G}$, which maps global identities GID to elements of $\mathbb{G}$.

3.2.2. Authority setup

In MA-ABE, each authority AA_i is responsible for managing a subset of attributes and performs a local setup to generate its public and private keys:

Choose $\alpha_k, y_k \in \mathbb{Z}_p$

$$PK_i = \{ e(g_1, g_1)^{\alpha_k}, g_1^{y_k} \}, \quad MSK_i = \{ \alpha_k, y_k \}$$

3.2.3. KeyGen

To create a key for a global identity GID for attribute i managed by an authority, the authority computes:

$$K_{i,GID} = g_1^{\alpha_k} \cdot H(GID)^{y_k}$$

3.2.4. Encryption

The encryption of a message M is performed according to an access policy (A, ρ) , which specifies the required attributes for decryption. The ciphertext is generated as:

$$\begin{aligned} C_0 &= M \cdot e(g_1, g_1)^s \\ C_{1,x} &= e(g_1, g_1)^{\lambda_x} \cdot e(g_1, g_1)^{\alpha_{\rho(x)} r_x} \\ C_{2,x} &= g_1^{r_x} \\ C_{3,x} &= g_1^{y_{\rho(x)} r_x} \cdot g_1^{\omega_x} \quad \forall x \end{aligned}$$

where s is a secret random value chosen from $\mathbb{Z}_p$, $\lambda_x = A_x \cdot v$, where A_x is the x th row of A , $\omega_x = A_x \cdot w$ where $w \in \mathbb{Z}_p^\ell$ is a random vector with 0 as its first entry, and $r_x \in \mathbb{Z}_p$ is a random chosen for each row A_x of A .

3.2.5. Decryption

The decryptor first computes $H(\text{GID})$. If the decryptor possesses the secret keys $\{K_{\rho(x), \text{GID}}\}$, then the decryptor proceeds as follows:

$$C_{1,x} \cdot \frac{e(H(\text{GID}), C_{3,x})}{e(K_{\rho(x), \text{GID}}, C_{2,x})} = e(g_1, g_1)^{\lambda_x} \cdot e(H(\text{GID}), g_1)^{\omega_x}$$

Next, the decryptor chooses constants $c_x \in \mathbb{Z}_N$ such that:

$$\sum_x c_x A_x = (1, 0, \dots, 0)$$

Then computes:

$$\prod_x (e(g_1, g_1)^{\lambda_x} \cdot e(H(\text{GID}), g_1)^{\omega_x})^{c_x} = e(g_1, g_1)^s$$

Finally, the message M is recovered as:

$$M = \frac{C_0}{e(g_1, g_1)^s}$$

3.3. Shamir Secret Sharing Scheme

Shamir's Secret Sharing Scheme (Shamir, 1979) is a threshold-based cryptographic method that splits a secret S into n shares such that any t or more shares ($t \leq n$) can reconstruct S , while fewer than t shares reveal no information about it.

The scheme is based on polynomial interpolation over a finite field $\mathbb{F}_p$, where p is a large prime. A random polynomial of degree $t - 1$ is constructed as:

$$f(x) = S + a_1x + \dots + a_{t-1}x^{t-1}, \quad a_i \in \mathbb{F}_p$$

Each share is a point $(x_i, f(x_i))$ for some $x_i \in \mathbb{F}_p^*$.

To recover S , any t shares are sufficient. The secret is reconstructed using Lagrange interpolation at $x = 0$:

$$S = \sum_{i=1}^t f(x_i) \prod_{\substack{j=1 \\ j \neq i}}^t \frac{x_j}{x_j - x_i} \mod p$$

This scheme ensures *information-theoretic security*, meaning that no knowledge about S can be obtained with fewer than t shares.

3.4. Object Security for Constrained RESTful Environments (OSCORE)

Object Security for Constrained RESTful Environments (OSCORE) (Selander et al., 2019) is a lightweight end-to-end security protocol for constrained IoT systems. Operating at the application layer, it protects Constrained Application Protocol (CoAP) messages using Authenticated Encryption with Associated Data (AEAD), independently of the transport layer (e.g., UDP, DTLS).

OSCORE derives cryptographic keys from a pre-shared secret or a key exchange protocol (e.g., EDHOC) and secures CoAP payloads, critical options, and headers.

Therefore, it guarantees confidentiality, integrity, authenticity, and replay protection, while supporting multicast communication via its extension, *Group OSCORE*.

4. Proposed access control scheme

In this section, we present our proposed scheme for controlling access to the execution of actions on IoT objects. Our solution enables users, who may belong to different authorities within a distributed Edge computing architecture, to securely interact with IoT objects by leveraging a combination of MA-ABE and a shared secret mechanism.

4.1. Architecture

In the proposed protocol, we consider an architecture composed of the following components.

4.1.1. Authorities

Multiple authorities may co-exist in our architecture. Each authority is responsible for managing a disjoint subset of attributes and issuing the corresponding public keys for these attributes. Moreover, it generates and distributes private keys to users possessing specific attributes. Authorities are assumed to be fully trusted entities in the system, and it is assumed that they do not collaborate with malicious users or leak cryptographic keys.

4.1.2. Edges nodes

Edge nodes act as intermediaries between IoT objects and users. Each node manages a group of IoT objects, enforces access policies defined by the object owners, processes user requests and issues short-lived access tokens. To ensure the legitimacy of access policies, edge nodes authenticate object owners using mechanisms such as digital certificates. Edge nodes are assumed to be honest but curious: they follow the protocol correctly but may attempt to access IoT objects beyond their control.

4.1.3. IoT objects

IoT objects perform specific actions requested by users. Each object is managed by an edge node, which enforces the access control policies defined by the object owner. These objects are considered untrusted: they may attempt to access other objects, leak information, or behave maliciously if compromised.

4.1.4. Users

Users interact with IoT objects to request the execution of specific actions. Each user is associated with one or more authorities, which assign them a set of attributes and issue corresponding decryption keys. These attribute-based keys are used by users to decrypt the access policies associated with actions on IoT objects, thereby proving their authorization to execute them. Users are considered untrusted: they may attempt to gain unauthorized access to objects or perform actions beyond their granted permissions.

4.1.5. Objects owners

Object owners define fine-grained access control policies that specify the conditions under which users are authorized to perform specific actions on IoT objects. These policies are based on users attributes and are enforced by the system before any action is carried out. Objects owners are considered as trusted entities within their respective domains and are responsible for defining the validity period of access tokens to ensure flexible and context-aware access control.

4.2. Threat model

In our protocol, we distinguish two different adversarial models where each model has a particular situation defined as follows:

4.2.1. The case where an attacker aims to compromise the token

In this threat model, we consider a polynomial-time adversary $\mathcal{A}$ interacting with a challenger encryption system. The adversary $\mathcal{A}$ is allowed to query the encryption of plaintext tokens under attribute-based access policies of its choice. At the challenge phase, $\mathcal{A}$ submits two challenge tokens (t_0, t_1) and a target access policy $\mathcal{P}^*$. The challenger then randomly selects a bit $b \in \{0, 1\}$ and generates the challenge ciphertext $\text{CT}^* = \text{Enc}_{\mathcal{P}^*}(t_b)$, which is returned to $\mathcal{A}$.

The adversary wins the security game if it correctly guesses the value of b with a probability non-negligibly greater than $1/2$. The encryption scheme is considered *IND-CPA secure* (Indistinguishability under Chosen Plaintext Attack) if no adversary $\mathcal{A}$ can achieve this with non-negligible advantage.

4.2.2. The case where an attacker aims to impersonate an authorized user

In this threat model, we consider a polynomial-time adversary $\mathcal{A}$ interacting with a challenger access control system. The adversary $\mathcal{A}$ is initially granted access to all credentials associated with its own identity GID in the system. It is allowed to submit these credentials to the challenger for any action A_{ij} it is authorized to perform. Upon verification, the challenger responds with *Access Allowed* if the credentials are valid.

During the challenge phase, the adversary $\mathcal{A}$ submits a modified set of credentials for the same action A_{ij} , but associates them with a different identity $GID' \neq GID$. The objective of the adversary is to manipulate the credentials such that the challenger responds with *Access Allowed* for the action A_{ij} under the forged identity GID' .

The adversary $\mathcal{A}$ wins the security game if it successfully gains access to A_{ij} using GID' , demonstrating a breach in the identity verification and credential validation mechanisms of the system.

4.3. Main idea

Our access control scheme relies on a one-to-many access token mechanism to regulate interactions with IoT objects. Tokens are generated by edge nodes and issued to IoT objects, along with a portion of a shared secret.

When a user requests to perform an action on an IoT object, the edge node managing that object provides an encrypted token. Encryption is performed using MA-ABE under an access policy defined by the object owner. The token is issued alongside the shared secret between the user and the object, enabling multiple users who satisfy the access policy to use the same token.

Once obtained, the token and shared secret can be reused for subsequent requests of the same action, reducing the need for repeated interactions with the edge node. IoT objects, meanwhile, do not perform heavy cryptographic operations, focusing only on executing authorized actions.

Note that in this work we address only the access control service. Therefore, any other security-related service falls outside the scope of this paper.

In the following sections, we present a detailed explanation of the main phases of our access control scheme. The key notation used throughout the solution are summarized in Table 2.

4.4. Access control scheme

Fig. 1 illustrates how the proposed access control scheme operates. In the following, we present the different execution phases of the proposed protocol.

4.4.1. System initialization

The system initialization phase consists of two sub-phases: (i) the setup of attribute authorities, during which each authority generates its public parameters and issues private decryption keys to users based on their assigned attributes; and (ii) the configuration of edge nodes, where objects owners define fine-grained access control policies using attributes managed by the authorities. These policies specify the exact conditions under which users are authorized to perform specific actions on IoT objects. The details of each sub-phase are described below.

4.4.1.1. Authority setup. In the proposed protocol, all attribute authorities agree on a cyclic group G of prime order N , generated by an element g , and a symmetric bilinear map $e : G \times G \rightarrow G_T$.

In addition, they define cryptographic hash functions $H : \{0, 1\}^* \rightarrow G$ to map identities to group elements G , $F1 : G_T \rightarrow \{0, 1\}^n$ to map elements from the group G_T to bit strings and $F2 : \{0, 1\}^n \rightarrow \{0, 1\}^n$ to hash the nonce used between IoT objects and users from n -bits to n -bits. These components constitute the global public parameters, shared among all entities in the system:

$$GP = \{G, G_T, g, e, H(\cdot), F1(\cdot), F2(\cdot)\} \quad (1)$$

Table 2
Notation table.

Notation	Description
G, G_T	Multiplicative cyclic groups of prime order N used in bilinear pairing-based cryptography.
N	A large prime number defining the order of the groups G and G_T .
g	A generator of the group G .
e	A bilinear pairing function $e : G \times G \rightarrow G_T$.
H	A cryptographic hash function mapping identities to group elements.
$F1$	A cryptographic hash function which maps elements from the group G_T to bit strings.
$F2$	A cryptographic hash function used to hash the nonce
λ	A security parameter defining the bit-length of cryptographic primitives.
GP	The Global public parameters generated during the setup phase.
α_k, y_k	The Master secret values held by authority i .
$\mathbb{Z}_N$	The finite field of integers modulo N .
SK_i	The Private key of authority i or user i .
PK_i	The Public key of authority i .
C_n	The Category i that groups a set of actions j .
CT_p	The Clear token associated with action i .
ET_q	The Encrypted token associated with action i .
$RV_{O_{ij}}$	Random value issued to object i for controlling access to action j .
$TS_{ij,GID}$	Secure token issued to user GID for performing action j on object i , also used to cache the corresponding $RV_{O_{ij}}$ via a One-Time Pad mechanism.
NS	The nonce used to ensure randomness and prevent replay attacks.
$SU1_i$	The Sub-share of the shared secret $SU_{ij,GID}$ held by object i .
$SU2_{ij,GID}$	The Sub-share of the shared secret $SU_{ij,GID}$ held by user GID .
$SU_{ij,GID}$	The Shared secret between user GID and object i for action j .
AA_i	The Attribute Authority i .
IDO_i	The Identifier of IoT object i .
O_i	The IoT object i .
U_{GID}	The User identified by global ID GID .
EN_m	The Edge node m responsible for managing access in a specific zone.
A_{ij}	The Action j of object i .
Enc_{MA-ABE}	Multi-authority attribute-based encryption function.
Dec_{MA-ABE}	Multi-authority attribute-based decryption function.
Enc_{sym}	Symmetric encryption function (e.g., AES) used for efficient encryption.

Each attribute authority AA_i , managing a set of attributes SA_i . Therefore, for each attribute $attr_k \in SA_i$ the authority chooses two random values $\alpha_k, y_k \in \mathbb{Z}_N$, and defines:

$$PK_{attr_k} = \{e(g, g)^{\alpha_k} \in G_T, \quad g^{y_k} \in G\} \quad (2)$$

$$MSK_{attr_k} = \{\alpha_k, y_k \in \mathbb{Z}_N\} \quad (3)$$

The master secret key MSK_{attr_k} is kept private by the authority, while the public key PK_{attr_k} is published and made available to other entities for encryption.

4.4.1.2. Edge node configuration. To ensure efficient access control and secure communication between IoT objects and users, the object owner first defines a set of categories C_n , $n \in \mathbb{N}^*$, $n \leq N$, each category represents a group of actions A_j , $j \in \mathbb{N}^*$, $j \leq N$ performed by IoT objects O_i , $i \in \mathbb{N}^*$, $i \leq N$.

For each category, the owner also specifies an access policy $\mathcal{P}_n$, $n \in \mathbb{N}^*$, $n \leq N$ based on the attributes issued by the authorities. Table 3 provides an example of the mapping between categories and their corresponding access policies.

Once the categories and their respective policies are established, each Edge node EN_m will receive the categories and policies corresponding to the group of objects under its management. This categorization optimizes the encryption process, reduces storage overhead,

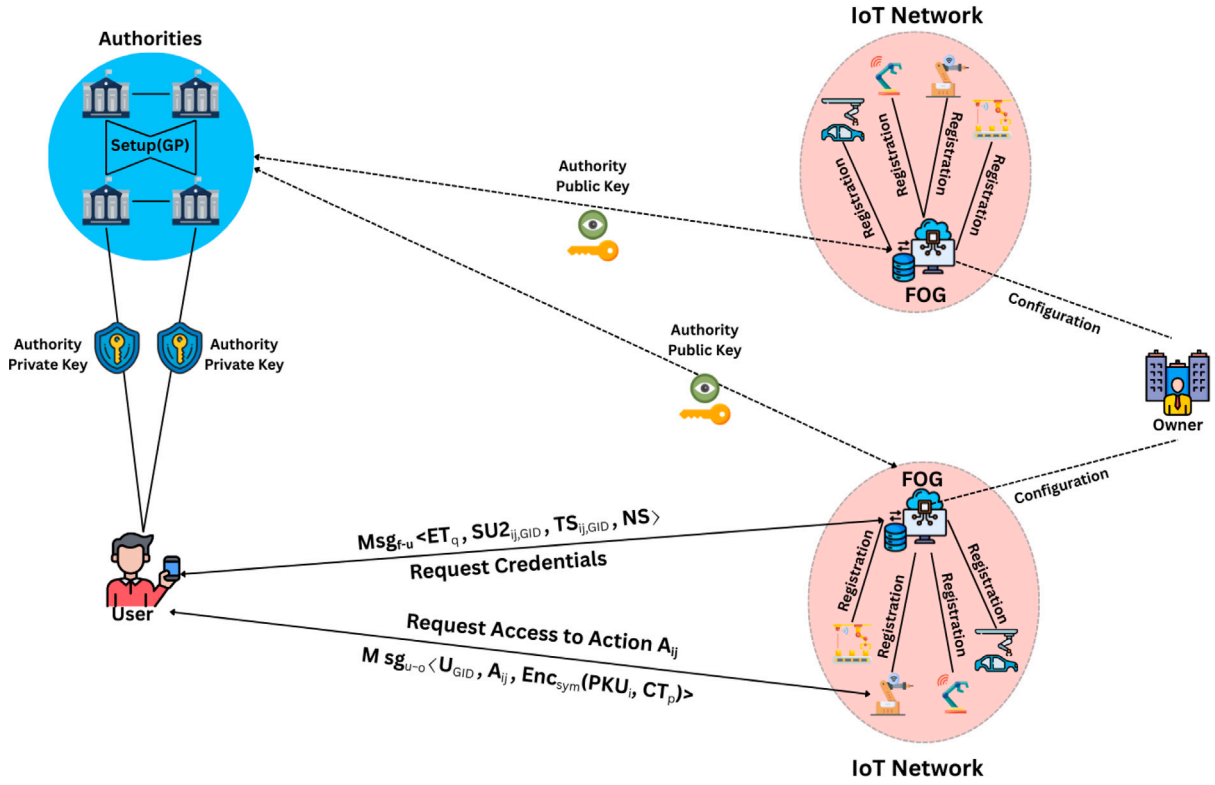


Fig. 1. Distributed IoT access control.

Table 3

Access control policies based on categories.

Category	Access policy	EXP TIME (h)
C_{n1}	$(Attr_{k1} \text{ AND } Attr_{k2}) \text{ OR } Attr_{k3}$	400 h
C_{n2}	$(Attr_{k3} \text{ AND } Attr_{k2}) \text{ OR } Attr_{k1}$	200 h
C_{n3}	$(Attr_{k1} \text{ AND } Attr_{k3}) \text{ OR } Attr_{k2}$	800 h

and ensures that each Edge node enforces only a minimal set of policies relevant to the IoT devices it manages.

The owner also defines an expiration time for the tokens, which will be managed by each Edge node to limit the valid period during which the token can be used.

4.4.2. IoT objects registration

In this phase, the IoT object O_i encapsulates the set of its executable actions A_{ij} in a message denoted $MAO\langle A1, A2 \dots A_j \rangle$. Then, it sends MAO to edge node EN_m over a secure channel relying on OSCORE.

Upon receiving the MAO message, EN_m extracts the executable actions A_{ij} of the object O_i , groups them by their corresponding categories, and then generates a random access token $CT_p \in \mathbb{G}_T$ for each group of actions.

This token is then encrypted according to the access policy P_n corresponding to its category, using the MA-ABE encryption method (detailed in Section 4.4). The resulting ciphertext is formalized as follows:

$$ET_q = \left\{ \begin{array}{l} C_0 = CT_p \cdot e(g_1, g_1)^s, \\ C_{1,x} = e(g_1, g_1)^{A_x} \cdot e(g_1, g_1)^{\alpha_{p(x)} r_x}, \\ C_{2,x} = g_1^{r_x}, \\ C_{3,x} = g_1^{y_{p(x)} r_x} \cdot g_1^{\omega_x} \quad \forall x \end{array} \right\} \quad (4)$$

Additionally, EN_m generates a random value $SU1_i \in \mathbb{Z}_N^*$ for object O_i along with set of random values $RVO_{ij} \in \{0, 1\}^n$ for each action A_{ij}

Finally, EN_m stores the object identifier IDO_i , $SU1_i$, and the sets $\{A_{ij}, ET_q, RVO_{ij}\}$ in the credentials table **cred_table**. Then sends a message Msg_{f-o} back to the IoT object O_i over a secure channel relying on OSCORE, in the following form:

$$Msg_{f-o} \langle IDO_i, SU1_i, \{A_{ij}, CT_p, RVO_{ij}\} \rangle \quad (5)$$

When the object O_i receives the message Msg_{f-o} , it stores the set of actions A_{ij} with their corresponding tokens CT_p and RVO_{ij} in the table **object_actions_cred_table**.

This phase ensures that each registered IoT object has a well-defined and protected set of executable actions while allowing the Edge node to control and secure access to resources.

4.4.3. Access to actions of objects

To request the execution of a specific action A_{ij} on an IoT object O_i , users have two phases of access. The first phase is executed with the Edge node if the user requests the execution of an action in object O_i for the first time, or after a certain time-out period (t). Then it will allow the user to obtain the credentials required for the execution of actions during the next interactions with the object.

The second phase occurs when the user has already acquired the credentials. It will then allow direct access to the object without passing through the Edge node.

4.4.3.1. First access phase. If a user U_{GID} wants to execute an action A_{ij} on an object O_i , they send an access request in the form of a message $\langle U_{GID}, A_{ij} \rangle$ to the EN_m , which manages the object O_i .

Upon receiving the access request, EN_m searches for the credentials corresponding to the requested action A_{ij} of the object O_i in the credentials table **cred_table**. If the requested action and the identity IDO_i of the object O_i have been stored in the table **cred_table**, EN_m will succeed to retrieve the corresponding encrypted token ET_q , the random value RVO_{ij} and $SU1_i$ associated with O_i and action A_{ij} . After that the EN_m uses $SU1_i$ to build a shared value based on the Shamir Secret Sharing scheme. First, it generates a random value $SU_{ij,GID} \in$

$\{0,1\}^n$, where $SU_{ij,GID} \in \mathbb{Z}_N^*$. Then, it computes the second share $SU_{2ij,GID}$ using a degree-1 polynomial:

$$f(x) = c_0 + c_1 \cdot x \mod p \quad (6)$$

where:

$$c_0 = SU_{ij,GID} \quad (7)$$

$$c_1 = (SU_{1i} - SU_{ij,GID}) \mod p \quad (8)$$

Finally, the second share $SU_{2ij,GID}$ for the user is computed at $x = 2$:

$$\begin{aligned} SU_{2ij,GID} &= f(2) \\ &= SU_{ij,GID} + (SU_{1i} - SU_{ij,GID}) \cdot 2 \mod p \\ &= 2 \cdot SU_{1i} - SU_{ij,GID} \mod p \end{aligned}$$

After that, EN_m obfuscates RVO_{ij} by making it unique for the user U_{GID} using a One-Time Pad, as shown below:

$$TS_{ij,GID} = RVO_{ij} \oplus SU_{ij,GID} \quad (9)$$

Next, EN_m generates a random session nonce $NS_{i,GID} \in \{0,1\}^n$, which will be used in future communications between O_i and U_{GID} .

Finally, EN_m stores the pair $\langle U_{GID}, NS_{i,GID} \rangle$ in **edge_user_nonce_table** and sends the following message to the user U_{GID} over a secure OSCORE-protected channel:

$$Msg_{f-u} \langle ET_q, SU_{2ij,GID}, TS_{ij,GID}, NS_{i,GID} \rangle \quad (10)$$

When the user U_{GID} receives the message from EN_m , they extract ET_q from the message and attempt to decrypt it using their private keys $\{K_{p(x),GID}\}$ issued by the authorities AA_I , as shown below:

$$C_{1,x} \cdot \frac{e(H(GID), C_{3,x})}{e(K_{p(x),GID}, C_{2,x})} = e(g_1, g_1)^{\lambda_x} \cdot e(H(GID), g_1)^{\omega_x} \quad (11)$$

Finally, if the user possesses a satisfying set of keys associated with the same identity GID, the terms $e(H(GID), g_1)^{\omega_x}$ cancel out, since the ω_x values are shares of 0. As a result, the clear token CT_p can be successfully recovered as:

$$CT_p = \frac{C_0}{e(g_1, g_1)^s} \quad (12)$$

Therefore, if the decryption is successful, the user obtains the clear token CT_p .

After that, the user U_{GID} groups $TS_{ij,GID}$ and $SU_{2ij,GID}$ into a packet PKU in the form $PKU \langle TS_{ij,GID}, SU_{2ij,GID} \rangle$ and encrypts it using CT_p as a symmetric key and $NS_{i,GID}$ as a nonce, under a symmetric encryption scheme. The resulting ciphertext is then sent to the object O_i in the following format:

$$Msg_{u-o} \langle U_{GID}, A_{ij}, Enc_{sym}(PKU, F1(CT_p), NS_{i,GID}) \rangle \quad (13)$$

When the IoT object O_i receives an execution request Msg_{u-o} from the user U_{GID} , it searches in the table **object_actions_cred_table** for the clear token CT_p , the pair $(1, SU_{1i})$, and RVO_{ij} corresponding to action A_{ij} . Next, it contacts the Edge node EN_m over a secure channel to request the nonce $NS_{i,GID}$ associated with U_{GID} and O_i .

- If no nonce is returned by EN_m , the access process fails, and O_i responds to the user with the message "Access Denied".
- Otherwise, O_i uses $NS_{i,GID}$ returned by EN_m and the clear token $F1(CT_p)$ to decrypt the received message Msg_{u-o} and retrieve $TS_{ij,GID}$ and $SU_{2ij,GID}$. If $F1(CT_p)$ does not correspond to the actions A_{ij} , or if it is expired, the decryption process fails and the access request will be rejected.

After that, if the decryption process succeeds, O_i uses the retrieved share $SU_{2ij,GID}$ and its share SU_{1i} corresponding to action A_{ij} to

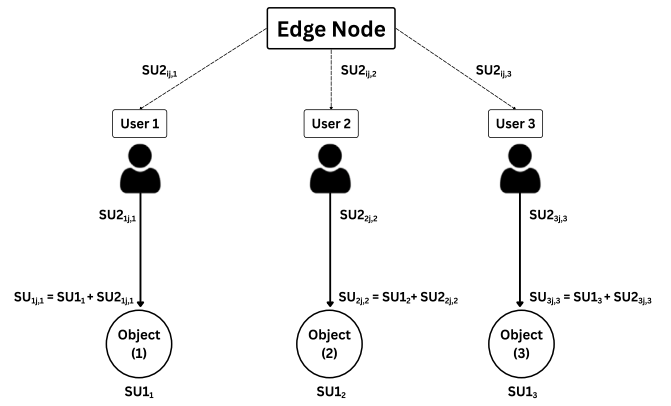


Fig. 2. Distribution of secret components between users and IoT objects.

reconstruct the secret $SU_{ij,GID}$ using Lagrange interpolation at $x = 0$ as:

$$SU_{ij,GID} = f(0) = SU_{1i} \cdot \lambda_1 + SU_{2ij,GID} \cdot \lambda_2 \mod p, \quad (14)$$

where the Lagrange coefficients are defined as:

$$\lambda_1 = \frac{-x_2}{x_1 - x_2}, \quad \lambda_2 = \frac{-x_1}{x_2 - x_1}, \quad (15)$$

with $x_1 = 1$ and $x_2 = 2$.

Thus, the reconstructed secret simplifies to:

$$SU_{ij,GID} = 2 \cdot SU_{1i} - SU_{2ij,GID} \mod p. \quad (16)$$

Fig. 2 illustrate how the secret $SU_{ij,GID}$ is distributed between the user and the IoT object using Shamir's Secret Sharing Scheme.

Upon reconstructing $SU_{ij,GID}$, the object O_i uses it to retrieve RVO'_{GID} using a One-Time Pad, as shown below:

$$RVO'_{GID} = TS_{ij,GID} \oplus SU_{ij,GID} \quad (17)$$

After that, it compares RVO'_{GID} with its pre-stored RVO_{ij} corresponding to action A_{ij} . If both values match, O_i executes the requested action A_{ij} and responds to the user U_{GID} with the message **<Access Granted>**. Otherwise, the request is rejected, and O_i responds with the message **<Access Denied>**.

Subsequently, if access is granted, both O_i and U_{GID} update their $NS_{i,GID}$ values using a one-way hash function, as shown below:

$$NS_{i,GID} = F2(NS_{i,GID}) \quad (18)$$

Finally, upon successful completion of the access process, the object O_i stores the pair $\langle U_{GID}, NS_{i,GID} \rangle$ in **object_user_nonce_table** and the user U_{GID} stores the values $TS_{ij,GID}$, $SU_{2ij,GID}$, $NS_{i,GID}$, and CT_p in **object_action_cred_table**.

4.4.3.2. Subsequent access. During subsequent access attempts, U_{GID} can directly present the stored credentials to O_i as proof of a valid session, without the need to contact the edge node EN_m again. For this purpose, U_{GID} sends the message Msg_{u-o} as shown in the first access phase, in the following form:

$$Msg_{u-o} \langle U_{GID}, A_{ij}, Enc_{sym}(PKU_i, F1(CT_p), NS_{i,GID}) \rangle \quad (19)$$

Upon receiving Msg_{u-o} , O_i search for the clear token CT_p , the pair SU_{1i} and the pre-stored RVO_{ij} from the table **object_actions_cred_table** using action A_{ij} , and the value $NS_{i,GID}$ of the user U_{GID} from the table **object_user_nonce_table**.

Then, O_i decrypts the packet PKU_i (included in Msg_{u-o}) using the symmetric key CT_p and the nonce $NS_{i,GID}$ of the user U_{GID} , in order to retrieve $TS_{ij,GID}$ and $SU_{2ij,GID}$.

Afterward, O_i uses the retrieved share $SU_{2ij,GID}$ and its share SU_{1i} corresponding to action A_{ij} to reconstruct the secret $SU_{ij,GID}$ using

Lagrange interpolation at $x = 0$, as shown in the first access phase. Then, it uses $SU_{ij,GID}$ to retrieve RVO' via a One-Time Pad:

$$RVO'_{GID} = TS_{ij,GID} \oplus SU_{ij,GID}$$

Next, O_i compares RVO'_{GID} with its pre-stored RVO_{ij} corresponding to A_{ij} . If both values match, O_i executes the requested action A_{ij} and responds to U_{GID} with the message <Access Granted>. Otherwise, the request is rejected, and O_i responds with <Access Denied>.

Finally, if access is granted, both O_i and U_{GID} update their $NS_{i,GID}$ values using a one-way hash function, as shown in Eq. (18).

This phase eliminates the need for communication with the edge node EN_m during subsequent accesses for both the object O_i and the user U_{GID} . By leveraging pre-validated credentials, the system significantly reduces computational overhead, minimizes latency, and enhances the overall efficiency of the authentication process.

5. Security analysis

In our solution, an attacker may follow different strategies to bypass the access control mechanism. He may try to forge incorrect credentials to deceive the entities in the protocol, aiming to gain unauthorized access and perform restricted actions. He also may attempt to impersonate legitimate entities. In the following, we show that breaking our protocol would require the attacker to solve one or more well-known hard problems from the literature, which provides strong assurance of its security.

5.1.1. Security assumptions

• Assumption 1:

Let Enc_{sym} be a symmetric encryption function defined as $Enc_{sym} : \{0,1\}^* \times \{0,1\}^n \rightarrow \{0,1\}^*$. A polynomial-time adversary has a negligible advantage finding $K \in \{0,1\}^n$, only given $CT = Enc_{sym}(M, K)$.

• Assumption 2:

Let S_{AEAD} the authenticated encryption with associated data scheme used by OSCORE protocol. A polynomial-time adversary has a negligible advantage compromising the confidentiality, the integrity and the authenticity of message M , only given $CT = S_{AEAD}(M)$.

• Assumption 3:

Let G be a cyclic group of order N and g its generator. Let e be a bilinear map defined as $e : G \times G \rightarrow GT$. Finding the value $k \in \mathbb{Z}_p^*$ given g and $e(g, g)^k$ requires solving the problem of discrete logarithm that is assumed to be difficult in G .

• Assumption 4:

Let G be a cyclic group of prime order N with generator g . Given g^a and g^b for unknown random exponents $a, b \in \mathbb{Z}_N^*$, computing the value g^{ab} is assumed to be hard in G .

• Assumption 5:

Let G be a cyclic group of prime order with generator g , and let $e : G \times G \rightarrow GT$ be a bilinear map. A polynomial-time adversary has negligible advantage in recovering $M \in G_T$, given a ciphertext $CT = M \cdot e(g, g)^s$ and the public parameters generated during the setup phase of MA-ABE (defined in Section 3.B) over G .

• Assumption 6:

Let $\mathbb{F}_p$ be a finite field of prime order p , and let $s \in \mathbb{F}_p$ be a secret shared using a (t, n) -threshold Shamir Secret Sharing scheme. Finding the value s given fewer than t shares $\{(i, f(i))\}$ requires reconstructing a degree- $(t-1)$ polynomial, which is assumed to be infeasible over $\mathbb{F}_p$.

5.1.2. Proof of security

• Token compromise:

There are two situations that can be distinguished in the case an attacker aims to get unauthorized access to objects and thus trigger the execution of an action A_{ij} on an object O_i .

In the first situation, the adversary Adv_1 is an external entity that is completely unknown to the Edge nodes but still has access to the public information of the network and the messages exchanged in the network. One of this information can be the list of attributes and their associated public keys. To reach its objective, this adversary first needs to find the value of the token CT_p used by object O_i to establish access right proof. The value CT_p only appears in two messages. The first one is $M_{sg_{f-o}}$ sent by the EN_m to the object O_i . It also appears in an attribute-based encrypted format ET_q in message $M_{sg_{f-u}}$.

We recall that the token $M_{sg_{f-o}}$ is encrypted using an AEAD scheme of OSCORE and thus as a part of this message, the token CT_p is also encrypted by the same scheme. Furthermore, before the Edge Node EN_m sends the message $M_{sg_{f-u}}$ to another user, the clear token CT_p is first encrypted using an attribute-based access policy then the resulting ciphertext ET_q is encrypted using an AEAD scheme of OSCORE. The easiest situation for an adversary in these cases is to target the OSCORE message $M_{sg_{f-o}}$ as it involves one decryption using an AEAD scheme. If the adversary Adv_1 has an algorithm that succeeds in getting CT_p in a polynomial time, given only $M_{sg_{f-o}}(IDO_i, SU1_i, \{A_{ij}, CT_p, RVO_{ij}\})$, it can then be used as a sub-routine to break **Assumption 2**.

In the second situation, the adversary Adv_2 is an internal user of the network but does not have the required attributes to execute an action A_{ij} in the object O_i . Therefore, this adversary will also try to get the value CT_p then request the execution of the action A_{ij} . Unlike the adversary Adv_1 , the adversary Adv_2 can actually request the OSCORE messages $M_{sg_{f-u}}$ from the edge node and decrypt them as it is an internal user of the network. Therefore, he can also try to get the value of CT_p given ET_q .

In that case, the adversary Adv_2 will try to use its private keys (which are not supposed to allow the decryption of ET_q) and the public keys to retrieve CT_p . Let us suppose that Adv_2 is missing only one attribute $Attr_k$ needed to satisfy the access policy. The adversary will then aim to obtain the corresponding private key $K_{k,GID} = g_1^{a_k} \cdot H(GID)^{y_k}$ given a set of private keys $\{K_{i,GID} \mid i \neq k\}$ and public values $\{e(g_1, g_1)^{a_k}, g_1^{y_k}\}$.

To achieve this, the adversary must first compute a_k from $e(g_1, g_1)^{a_k}$, which would break **Assumption 3**. Additionally, the adversary would need to compute $H(GID)^{y_k}$ from known values $\{H(GID)^{y_i}\}$ and g^{y_k} using a polynomial time algorithm. If this algorithm exists, then we can easily build an adversary Adv_3 that breaks **Assumption 4** through the following steps:

- (1) Adv_3 receives the input value (g^a, g^b) .
- (2) Adv_3 puts $GID = g^a$ and chooses a random value $r \in \mathbb{Z}_p$.
- (3) Adv_3 calls Adv_2 with GID' and g^b as input.
- (4) Given GID' and g^b , Adv_2 will output GID^b if it has a non-negligible probability to do so.
- (5) Finally, Adv_3 returns GID^b which is actually equal to $(g^a)^b = g^{ab}$ and breaks **Assumption 4**.

The adversary Adv_2 may also try to target the ciphertext ET_q and try to get CT_p given $C_0 = CT \cdot e(g_1, g_1)^s$ using a polynomial-time algorithm. If such algorithm exists, it can then be used as a sub-routine to break assumption **Assumption 5**.

Finally, Adv_2 can also try to target the messages $M_{sg_{u-o}}$ sent by authorized users. The aim in that case will be to find CT_p and the nonce $NS_{i,GID}$ given $M_{sg_{u-o}}$ in a polynomial time. Similarly, if such algorithm exist, it can then be used as a sub-routine to break assumption **Assumption 1**.

- **User impersonation:**

There are three main situations that can be distinguished in the case where an adversary tries to impersonate a specific GID_v . In the first situation, the adversary is assumed to have required access right and thus has the attributes required to satisfy the access policy and get access to the value CT_p . However, this adversary tries to fool the object by providing the credentials of another entity and thus impersonate its identity GID_v . To succeed, the adversary first needs to forge the packet PKU . Therefore, he needs to come up with two values TS_{i,j,GID_v} , and $SU2_{i,j,GID_v}$ given $TS_{i,j,GID_{adv}}$ and $SU2_{i,j,GID_{adv}}$. Let us recall that $SU2_{i,j,GID_v} = 2 \cdot SU1_i - SU_{i,j,GID_v}$, while $SU2_{i,j,GID_{adv}} = 2 \cdot SU1_i - SU_{i,j,GID_{adv}}$. As we can notice, $SU2_{i,j,GID_{adv}}$ is computed using a random value $SU1_i$ specific for each object in addition to a random value $SU_{i,j,GID_{adv}}$ specific for each user and each action. Both random values are only known by the edge node and object O_i but the result $SU2_{i,j,GID_v}$ do not provide any indication on the targeted value SU_{i,j,GID_v} . These values are then obviously of no use to the adversary and thus its only option is to brute force the value $SU2_{i,j,GID_v}$ over Z_q^* .

In the second situation, the adversary may try to replay a message $M_{sg_{u-o}}(t)$, used for access request at instant (t) , to get access at instant $(t+1)$. Since the object O_i updates the nonce NS_{i,GID_v} after each access, the message $M_{sg_{u-o}(t+1)}$ will be encrypted using CT_p and the nonce $NS_{i,GID_v(t+1)} = H(NS_{i,GID_v(t)})$. Therefore, replaying the same message will cause a decryption failure at the object level and thus do not allow to grant access to the action A_{ij} . The only option left then will be to deduce NS_{i,GID_v} given $M_{sg_{u-o}(t)}$ which leads to breaking **Assumption 2** as shown above. In the third situation, the adversary is assumed to be a malicious object O_i and thus is supposed to have access to the values TS_{i,j,GID_v} , CT_p , NS_{i,GID_v} , $SU1_i$, $SU2_{i,j,GID_v}$ and $RVO_{i,j}$. It can also deduce the value SU_{i,j,GID_v} . In that case its purpose is to use these elements to execute an action on an object O_t and thus impersonating GID_v . This attack will succeed if O_i manages to deduce SU_{i,j,GID_v} using Shamir's secret reconstruction and NS_{i,GID_v} . However, the elements in possession of object O_i (i.e. $SU1_i$ and $SU2_{i,j,GID_v}$) allow only the reconstruction of SU_{i,j,GID_v} and are not related to SU_{i,j,GID_v} at all. Therefore, it will require the adversary to reconstruct a secret SU_{i,j,GID_v} with fewer shares (zero shares instead of two) and thus breaking assumption **Assumption 6**.

6. Application and performance evaluation

In this section, we present the application of our access control protocol within Industry 4.0 Smart Manufacturing Systems (SMSs). These systems represent a transformative shift in industrial production, characterized by full integration, real-time collaboration, and adaptability across the manufacturing ecosystem. According to the National Institute of Standards and Technology (NIST), smart manufacturing involves systems capable of dynamically responding to changes in production environments and customer demands (Kusiak, 2018). This is because these systems are designed to integrate products, operations, and business systems across multiple factories, distribution centers, enterprises, and supply chains (Yang et al., 2019).

Although this architectural vision facilitates collaborative and distributed production, it also highlights the critical need for secure, scalable, and cross-domain access control mechanisms. To address this need, several functional requirements must be considered. First, the proposed solution should support an architecture that includes (1) multiple smart factories, in which (2) smart equipment executes specific manufacturing tasks requested by (3) users. The access to these tasks is regulated through predefined access policies. Finally, the data related to access credentials of each subset of smart equipment is managed by an (4) edge node (Stouffer et al., 2023; Zheng et al., 2018).

As described in Section 4, the architecture adopted in our access control protocol aligns with the one adopted in SMSs. Specifically, our protocol assumes multiple independent authorities, each managing security within its own domain. In the case of SMS, the factories will act as authorities. Each factory will then define a set of attributes representing the users under its management.

First, the factories initialize the security system by executing the setup phase (Section 4.4.1.1), which generates the public parameters and the global set of attributes used in the system. Following the setup, each factory generates and distributes the private decryption keys to its users based on the attributes assigned to them. These users can be either human operators or smart equipment.

Next, each factory configures the access policies in its own network. We assume that this task is managed by a security administrator that executes the configuration phase (Section 4.4.1.2). This will define the categories grouping the actions each equipment is authorized to perform. It will also allow to initialize the edge servers with the access policies that govern conditions for accessing smart equipment. Since our protocol provides a multi-authority cryptographic access control mechanism, the access rights can be granted to any user who satisfies the access policy, regardless of their factory of origin.

At this point, the smart equipment must proceed with the registration phase (described in Section 4.4.2) by interacting with the edge node. Each smart equipment sends the list of actions it is capable of performing to the edge server. The edge server will then generate the corresponding credentials required to authorize those actions and returns them to the smart equipment.

Our access control protocol considers two scenarios to perform an action on a specific smart equipment. In the first scenario, we assume that the user either interacts for the first time with the smart equipment, or his access credentials have expired. In both cases, the user needs to run the access procedure defined in Section 4.4.3.1. This procedure will allow the user to prove his right to access by successfully decrypting the attribute-based ciphertext sent by the edge node as a challenge. The acquired access right, taking the form of an access token according to our protocol, can grant access during the current request and can also be reused for a limited time period. The expiration time is, in that case, set by the administrator during the configuration phase.

In the second scenario, the user has already a valid token for a given action. Therefore, he will rather run the access procedure defined in Section 4.4.3.2. This will allow for a direct and fast interaction with the smart equipment, without passing through the edge node, as long as the token is not expired.

In this scenario, the user and the IoT object must synchronize their nonces at the end of each access request, as shown in Section 4.4.3.2. However, if a network disruption occurs during communication at the end of the access control phase, a nonce desynchronization may arise between the two parties. To ensure robust synchronization in such cases, we introduce a counter-based mechanism inspired by the synchronization strategy used in OSCORE (Selander et al., 2019).

In this mechanism, a shared counter between the user and the object records the number of hash iterations applied to the evolving nonce. After each successful access, both the user and the object increment this counter following the hash update of the nonce. During subsequent access attempts, the user must in that case include the counter in the access-request message $M_{sg_{u-o}}$ (Section 4.4.3.2). Upon receiving this message, the object first compares the counter received from the user with its locally stored value. If a mismatch is detected, the object triggers a resynchronization procedure in which both parties repeatedly hash the nonce until they reach the highest counter value among them. This ensures consistent and secure nonce alignment even in the presence of network disruptions.

The major phases defining the application of our protocol to the SMS application use case are outlined in the sequence diagram presented in Fig. 3.

Table 4
Emulation parameters.

Parameter	Protocol/Platform
Operating System	Ubuntu
Platform	VirtualBox/Contiki cooja
Processor	Intel(R) Core(TM) i7-9750H 2.60GHz
Performance	5-core processor and 6 GB RAM
Protocol for Communication	CoAP/RIME
Protocol for Key Exchange	EDHOC
Protocol for Secure Channel	OSCORE under AES-CCM
One-Way Hash Function	SHA-256
Token Protection	MA-ABE
Libraries	Charm-Crypto, AI-CoAP, Hashlib
Mote type	Zolertia Z1 (MSP430)

To measure the performance of our protocol and provide a comparison with other access control approaches, we implemented an access control simulation suitable for the SMS application use case.

The computational cost of the different phases of our protocol have been evaluated on virtual machines according to the following scenario:

- **Factories (Authorities)** were emulated as a program that starts generating the security system (the system parameters and attributes), publishes public keys, and issues private keys for users.
- **Edge Servers** was emulated as a program that receives requests from IoT objects to generate credentials for their actions, processes users requests to obtain credentials for objects actions, and responds to objects requests to retrieve session nonce of users.
- **IoT objects** were emulated as a program that request the Edge node to generate credentials for their actions and also processes user requests to access these actions.
- **Users** were emulated as a program that requests credentials for objects actions from the Edge node and also sends access requests to perform specific actions on IoT objects.

The simulation was carried out on a single virtual machine running an Intel(R) Core(TM) i7-9750H 2.60 GHz processor, allocated 5 virtual cores and 6 GB of RAM. The implementation consists of four programs written in Python 3. Each program representing one of the above entities runs independently in its own thread. These programs communicate with each other over a local network using the CoAP protocol. In addition, communication between these programs was secured using the OSCORE and EDHOC protocols.

To implement the attribute-based encryption component of our protocol, we used the **abenc_maabe_rw15** library from the **Charm-Crypto** framework. Additionally, the CoAP and OSCORE protocols were implemented using the **AIOCOAP** library.

Furthermore, the energy consumption on IoT devices was evaluated using the Contiki-Cooja simulator (Dunkels et al., 2004; Osterlind et al., 2006). In our simulation scenario, we used the RIME protocol to handle communication between the edge node, sensors, and user devices via broadcast and multicast.

Additionally, all communications between IoT devices and the other system components were encrypted using the AES-CCM mode, implemented via the **CCM-STAR** library. Energy-related results were collected using the **ENERGEST** library. It is important to note that our evaluation does not account for the energy consumption during the execution of the tasks themselves, as the scope of our protocol is limited to the access control service. Therefore, we focus exclusively on the energy and time required to grant or deny access requests.

The main parameters used in this emulation are summarized in Table 4.

6.1. Evaluation of edge performance and user devices

In this section, we evaluate the performance of our protocol at the edge and user levels.

6.1.1. Analysis of network initialization time

Fig. 4 illustrates the time required by the edge node to initialize the network under five different scenarios, each involving an increasing number of participating IoT objects. In all scenarios, a single edge node processes registration requests, with the load modeled so that it receives one request per second from each IoT object. The number of objects used to initialize the network is increased incrementally in each scenario, ranging from 20 to 100. Additionally, the size of the access policy varies from 5 to 30 attributes. From Fig. 4, we observe that the time needed for the Edge node to initialize the network increases linearly as both the number of sensors and the size of the access policy grow. In the worst-case scenario, where the number of sensors reaches 100 and the access policy consists of 30 attributes, the initialization process takes approximately 40 s. This increase in initialization time can be explained by the tasks that the Edge node must perform. Specifically, for each group of actions, the Edge node needs to generate a token $CT_p \in \mathbb{G}_T$ and encrypt it using the MA-ABE scheme. This encryption involves multiple pairing operations, whose number increases with the complexity of the access policy. Furthermore, the Edge node must generate two random values, $SU1_i$ and RVO_{ij} for each action. However, when the number of sensors requesting registration simultaneously is limited and the number of attributes in the access policy remains moderate, the initialization time remains reasonable. As shown in Fig. 4, when approximately 20 sensors are involved and the access policy includes fewer than 25 attributes, the initialization time stays below 5 s.

6.1.2. Analysis of the registration time of IoT objects into the network

Fig. 5 illustrates the waiting time required for an IoT object to register on the network across five scenarios, each with an increasing number of tasks sent by the object. In every scenario, a single object sends registration requests to the edge node, which manages the registration process. The number of tasks increases incrementally from 4 to 20. Additionally, the size of the access policy varies between 5 and 30 attributes depending on the scenario. From Fig. 5, we observe that the registration time for the IoT object increases linearly with both the number of actions the object can perform and the size of the access policy.

This increase in registration time can be explained by the operations performed by the Edge node. As shown in the previous analysis, for each group of actions, the Edge node must generate a token $CT_p \in \mathbb{G}_T$ and encrypt it using the MA-ABE scheme. This encryption process includes a number of pairing operations that increase as the access policy becomes more complex, particularly when more attributes are involved. However, when the number of actions per object is small and the access policy remains moderate, the registration time becomes more acceptable. As shown in Fig. 5, when the number of tasks is around 4 and the access policy includes fewer than 25 attributes, the registration time for each object remains below 1 s.

6.1.3. Analysis of access request time

Fig. 6 illustrates the time required for a user to gain access to a specific action on an IoT object. The red curve represents the time needed for the first access, when the user does not yet possess the required credentials. In contrast, the blue curve shows the time for subsequent access, when the user already holds the necessary credentials. The test scenario consists of a single edge node and a single object, and the results present the average access time for issuing 40 access requests by a user at a rate of one request per second in both scenarios.

As shown in Fig. 6, when the user lacks the required credentials, the response time increases linearly with the size of the access policy. In the worst case, when the policy includes 30 attributes, the response time reaches approximately 1.5 s. This increase is mainly due to the cryptographic operations performed during the first access. In this phase, the user must contact the Edge Server to request the credentials of the action A_{ij} . The Edge Server locates the encrypted token ET_q

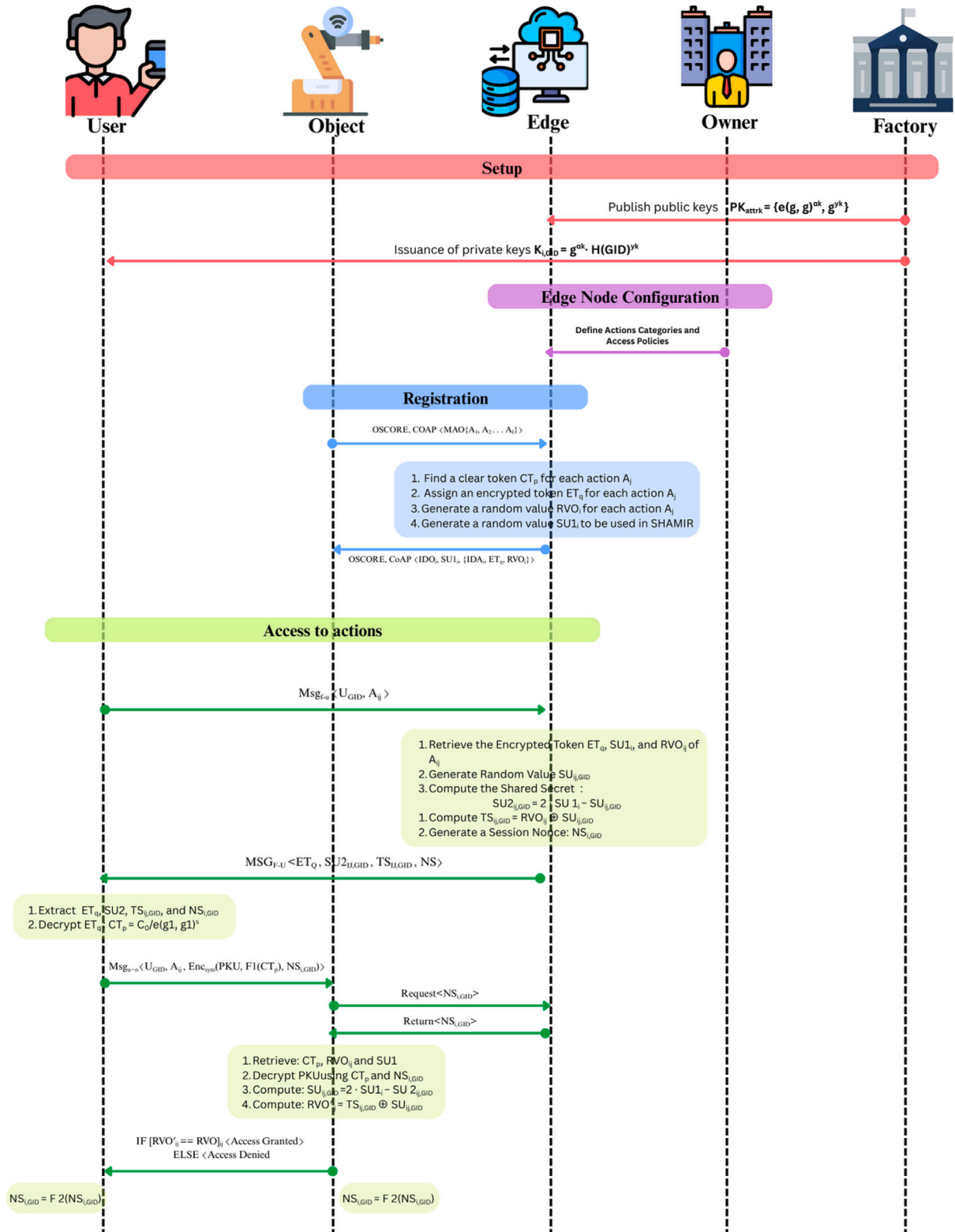


Fig. 3. The major sequences executed in our scheme.

corresponding to the requested action A_{ij} , retrieves the associated values RVO_{ij} and $SU1_i$, generates a random value $SU_{ij,GID}$, computes $SU2_{ij,GID}$ using the Shamir Secret Sharing scheme, and obscures RVO_{ij} using an One-Time Pad operation. All of these values are then returned to the user.

The user must then decrypt the encrypted token ET_q using the MA-ABE scheme to retrieve CT_p . This decryption involves a sequence of bilinear pairing operations, the number of which increases with the complexity of the access policy, specifically with the number of attributes it includes. Once CT_p is obtained, it is used to encrypt the

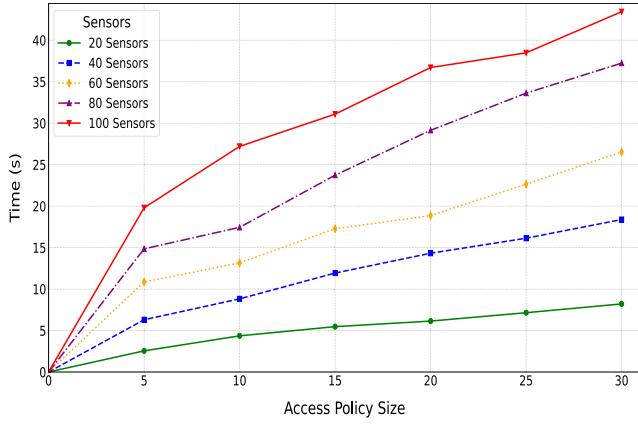


Fig. 4. Time required to initialize the network.

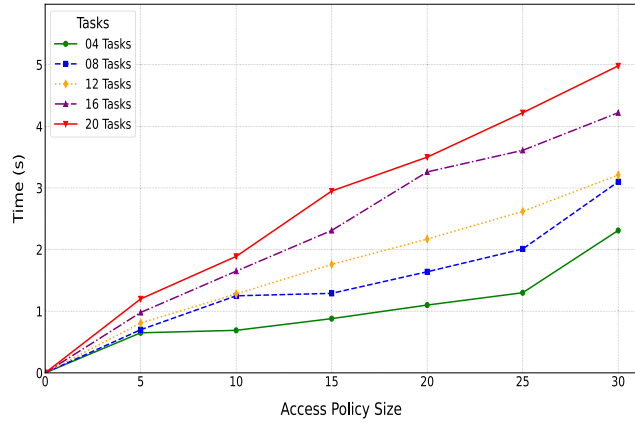


Fig. 5. Time required to register an object on the network.

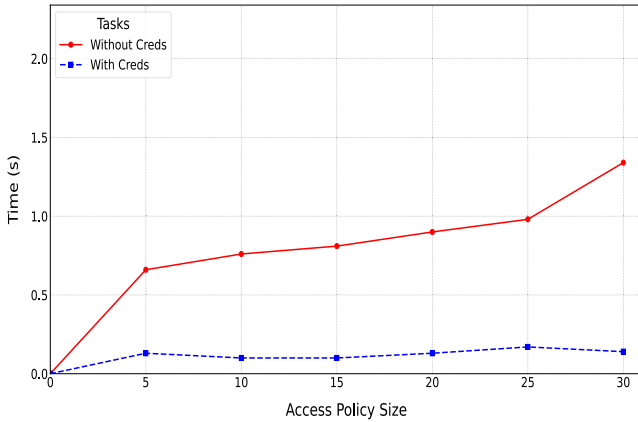


Fig. 6. Time required for a user to gain access to a specific action.

PKU, which contains $SU_{ij,GID}$ and $TS_{ij,GID}$. The user sends the PKU to the IoT object, which decrypts it using its own CT_p , reconstructs $SU_{ij,GID}$, and verifies RVO_{ij} to validate the access rights.

In subsequent accesses, when the user already has the necessary credentials to perform the action, the response time is consistently around 0.1 s, even as the size of the access policy increases. This is because the user no longer needs to contact the Edge Server or perform MA-ABE decryption, the required credentials are already available. The user can directly send the credentials to the IoT object, which verifies the access rights without additional processing. This highlights

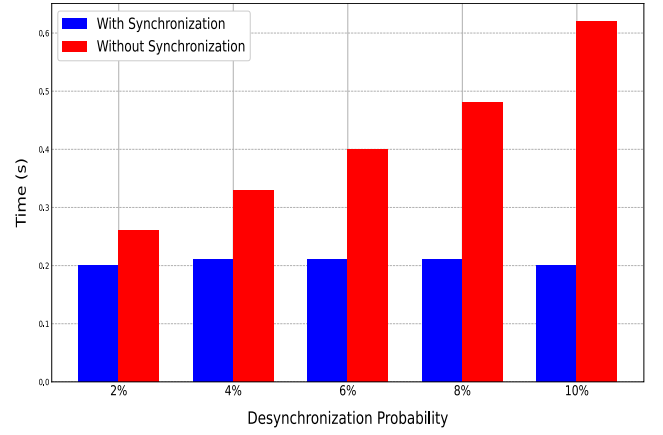


Fig. 7. Time required for a user to gain access to a specific action.

the efficiency of our approach, as it significantly reduces both energy consumption and response time once the initial credentials have been issued.

6.1.4. Analysis of access latency under nonce desynchronization

Fig. 7 illustrates the access latency experienced by a user attempting to execute an action on an IoT object during a subsequent access attempt, under different probabilities of nonce desynchronization between the two entities. The blue bars correspond to the scenario in which a synchronization mechanism is deployed, whereas the red bars represent the case where no such mechanism is available. The test setup consists of a single edge node and a single object, and the results present the average access time for issuing 40 access requests by a user at a rate of one request per second. The test was repeated 5 times, with the probability of nonce desynchronization increasing progressively from 2% to 10% across the tests. The number of attributes in the access policy was fixed at 15 for all tests.

As depicted in Fig. 7, in the absence of a synchronization mechanism, the access time (red bars) increases monotonically with the probability of desynchronization. This behavior is explained by the fact that, when the nonce held by the object does not match the nonce used by the user, the object is unable to decrypt the incoming request during the subsequent access. Such a failure forces the user to re-establish contact with the edge node in order to retrieve fresh credentials for the action A_{ij} . This procedure requires re-executing the entire access protocol, including all associated cryptographic operations, notably an MA-ABE decryption on the user side. As a consequence, the latency increases significantly as the desynchronization probability grows.

Conversely, when a synchronization mechanism is in place, the access time remains stable at approximately 0.2 s, independently of the desynchronization probability. In this case, the user does not need to request new credentials from the edge node, nor perform computationally expensive operations such as MA-ABE decryption. Instead, the user only performs a lightweight hash-based update of the nonce to realign with the value stored on the object. The refreshed credentials are then transmitted directly to the object, which validates the access rights without incurring additional processing overhead. This explains the near constant latency observed in the presence of the synchronization mechanism, highlighting its effectiveness in reducing both delay and computational cost.

6.2. Evaluation of sensor performance

In this section, we evaluate the performance of our protocol at smart equipment level.

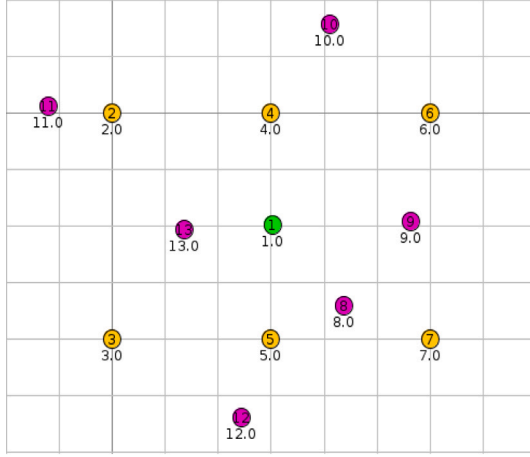


Fig. 8. Spatial approximation of objects to users.

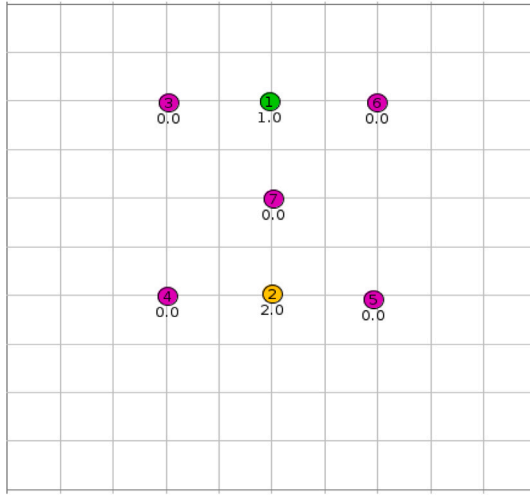


Fig. 9. Concurrent request handling by a single IoT object.

6.2.1. Energy consumption calculation

In the Contiki Cooja simulator, energy consumption is computed for different components such as CPU, Low Power Mode (LPM), Transmission, and Reception. The energy is calculated based on the usage time and the current consumed by each component.

6.2.1.1. Units of measurement. The following units are used in the computation:

- Supply Voltage (V_{SUPPLY}): Given in millivolts (mV), typically 3300 mV (3.3 V).
- Current Consumption ($I_{\text{CPU}}, I_{\text{LPM}}, I_{\text{TRANSMIT}}, I_{\text{LISTEN}}$): Given in milliamperes (mA).
- Usage Time ($t_{\text{cpu}}, t_{\text{lpm}}, t_{\text{transmit}}, t_{\text{listen}}$): Measured in milliseconds (ms), calculated using:

$$t_{\text{cpu, ms}} = \frac{\Delta_{\text{CPU}} \times 1000}{\text{RTIMER_SECOND}}$$

6.2.1.2. Scenario 1. In the first scenario as shown in Fig. 8, we used one Edge Server (Green bubble) to generate credentials and access control parameters, and 6 sensors (Orange bubble), each with three tasks to execute [A, B, C]. Each sensor can store the session nonce of 20 users and can serve 5 users simultaneously. We also used 6 users (Purple bubble), each located within the area of 3 sensors. Each user

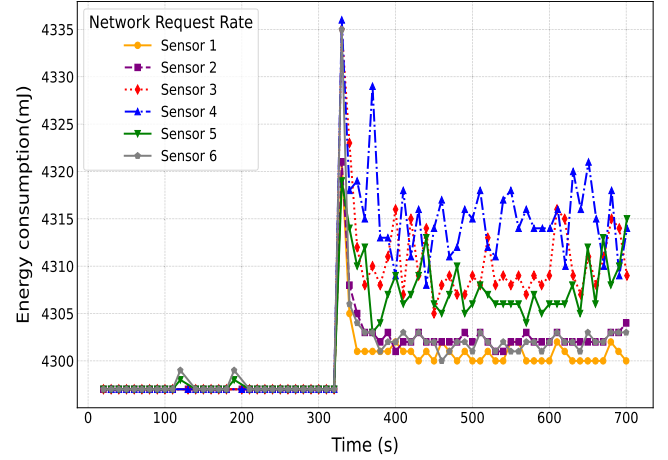


Fig. 10. Energy consumption versus network positioning.

calls a random sensor to execute one of the 3 tasks and waits for 3 s after each call. The network topology and the distribution of users are shown in Fig. 8, and the obtained results are presented in Fig. 10.

Fig. 10 shows the historical power consumption, we observe that during communication with the Edge Server to generate credentials, the power consumption is close to 2150 mJ. However, at second 300, when the sensors start receiving access requests from users, the power consumption increases to 4335 mJ. Afterward, it decreases to less than 4320 mJ. This behavior occurs because, upon receiving an initial request to execute an action, the sensor must communicate with the Edge Server to obtain the session nonce of the user. For subsequent access requests, the sensor only verifies the validity of the incoming data locally, without communicating with the Edge Server. If the number of incoming access requests remains constant, the power consumption stabilizes, as there are no additional factors, such as pairing operations or attribute authority that could affect the verification phase.

6.2.1.3. Scenario 2. In this scenario as shown in Fig. 9, we used a single Edge Server (Green bubble) to generate credentials and access control parameters, along with one sensor (Orange bubble) capable of executing three tasks: [A, B, C]. The sensor can store session nonce for up to 20 users and serve up to 5 users simultaneously.

Additionally, we analyze three cases as shown in Fig. 11:

- The green curve represents energy consumption when the IoT object receives one access request per second.
- The blue curve shows energy consumption when the object receives three access requests per second.
- The red curve illustrates energy consumption when the object receives five access requests per second.

From these curves, we observe that energy consumption is linearly correlated with the number of access requests received by the IoT object. As the number of requests increases, energy consumption also increases. However, when the number of requests remains constant, energy consumption stabilizes.

This behavior occurs because the IoT object does not perform intensive computational operations aside from symmetric encryption/decryption and basic comparisons, which consume minimal energy. With this architecture, we achieve energy efficiency while ensuring strong protection and security.

6.3. Storage evaluation

In this section, we analyze the storage overhead introduced by our protocol at both the Edge Node (EN_m) and IoT object levels.

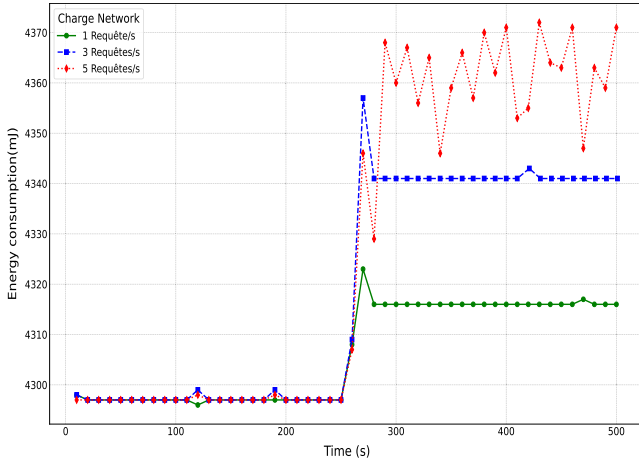


Fig. 11. Energy consumption versus number of requests.

Table 5
Storage cost of entities.

Entity	Storage cost
Edge Node	$N_O(3n + N_C(P + ET_q + n) + 2N_A n) + N_U(N_O \times 4n)$
IoT Object	$3n + N_A \times 2n + N_C \times n_T + N_U \times 3n$

The bit-length of ET_q is given by $|ET_q| = n_T + |X|(n_T + 2n_G)$, where n_G and n_T denote the bit-lengths of elements in groups G_1 and G_T , respectively, CT_p is of size n_T bits and $|X|$ is the number of attributes in the MA-ABE policy.

N_O , N_C , N_A , and N_U represent the number of IoT objects managed by the edge node, the number of categories per object, the number of actions per object, and the number of users registered at the edge node, respectively. $|P|$ denotes the bit-length of the access policy for each category.

6.3.1. Storage cost at the edge node (EN_m)

In our protocol, the edge node stores two types of information. The first type concerns the IoT objects. For each of the N_O objects managed by the edge node, it stores the object identity IDO_i , the OSCORE symmetric key, and the share of the secret SU_{1i} , each of length n bits, totaling $3n$ bits. Additionally, for each of the N_C action categories performed by the object, the edge node stores the access policy P of size $|P|$, the encrypted token ET_q of size $|ET_q|$, and the expiration time of size n , totaling $N_C(|P| + |ET_q| + n)$ bits. Furthermore, for the N_A actions associated with each object, the edge node stores the action identity A_{ij} and the corresponding random value RV_{O_i} , each of length n bits, $2N_A n$ in total.

The second type of information stored at the edge node concerns the users. For each user among the N_U registered users, and for every object that the user interacts with, the edge node stores the user identity $UGID$, the object identity IDO_i , the nonce $NS_{i, GID}$, and the synchronization counter, each of length n bits, totaling $N_U(N_O \times 4n)$ bits per pair. The complete storage overhead incurred at the edge node is summarized in Table 5.

This analysis reveals that the storage cost at the edge node primarily scales with the number of categories and actions per object, as well as the complexity of the MA-ABE policies encapsulated within the encrypted tokens (ET_q). By grouping actions under shared categories and associating a single encrypted token per category, the system effectively minimizes storage redundancy, reducing the need to store multiple identical tokens for actions with similar access policies. Moreover, this design leverages compact cryptographic representations and avoids replicating elements such as symmetric keys and Shamir secret shares, limiting them to a single instance per object and ensuring efficient use of memory resources. This approach balances fine-grained access control with practical storage constraints, making it suitable

for resource-constrained edge environments while maintaining robust security guarantees.

6.3.2. Storage cost at the IoT object

In our protocol, each IoT object stores two types of information. The first type concerns data related to the object itself and the actions it can perform. Each object must store its identity IDO_i , its share SU_{1i} from the secret sharing process, and the OSCORE symmetric key used to communicate with the edge node. Each of these elements has a size of n bits, for a total of $3n$ bits. Additionally, for the N_A actions associated with the object, it stores the identity of each action A_{ij} and the corresponding random value RV_{O_i} , each of length n bits, resulting in a total of $N_A \times 2n$ bits. Furthermore, for each of the N_C action categories, the object stores one clear token CT_p of size n_T bits (instead of one token per action), which contributes to reducing the overall storage overhead.

The second type of information stored at the IoT object concerns the users. For each user among the N_U users interacting with the object, it stores the user identity $UGID$, the nonce $NS_{i, GID}$, and the synchronization counter. Each of these fields has size n bits, yielding a total of $N_U \times 3n$ bits of user-related information. The complete storage overhead incurred at the IoT object is summarized in Table 5.

From this analysis, we observe that the storage cost on the object side does not increase with the number of performed actions, since the set of actions that each object can execute remains constant over time. Most of the storage overhead originates from the clear tokens (CT_p) in the target group G_T , which are inherently large due to cryptographic security constraints. However, the proposed protocol mitigates storage consumption in resource-constrained IoT devices by grouping actions into categories and associating a single token (CT_p) with each category. Moreover, fixed size elements such as the symmetric key, the object identity, and the partial Shamir shares are stored only once per device, thereby eliminating redundant data storage. Additionally, user-related information (e.g., identifiers, nonces, and counters) is maintained using small, fixed-size structures, enabling the IoT object to support multiple users without a significant increase in its storage footprint.

6.4. Performance comparison

Given that, to the best of our knowledge, our work is the first to address task-level access control on IoT objects using MA-ABE, direct comparisons with existing schemes are limited. Despite this, in this section, we present a comparison of the computational cost of our proposed access control scheme with several existing schemes from the literature. The comparison specifically evaluates the computational cost of the cryptographic operations performed by each protocol.

From Table 6, we observe that the computational cost of our scheme during the first access increases proportionally with the number of attributes q in the access policy. When q becomes sufficiently large, the schemes in Mandal et al. (2020) and Li et al. (2016) achieve better performance than our scheme during the first access. This is expected, since their computational costs are independent of the number of attributes, whereas the cost of cryptographic operations in our scheme grows linearly with the number of attributes q . However, this allows our protocol to offer better security features such as fine-grained access control and fully decentralized access control which is not the case of the other protocols.

Moreover, by comparing our scheme with (Zhang et al., 2021), we observe that our approach achieves significantly better performance during the first access. This improvement stems from the fact that our scheme performs considerably fewer pairing operations, which are known to be the most computationally expensive primitives in bilinear-pairing cryptography.

Regarding subsequent accesses in our scheme, it is clear that our protocol outperforms (Mandal et al., 2020), (Li et al., 2016), and (Zhang et al., 2021). This performance gain is due to the fact that, during

Table 6
Comparison of computation cost.

Scheme	Cost
Our Scheme in First Access	$3qM + (1 + 2q)P + r$
Our Scheme in Subsequent Access	r
Mandal et al. (2020)	$14M + r$
Li et al. (2016)	$3M + 6P + 1E$
Zhang et al. (2021)	$9M + 6qP + 3E$

We denote by M the point multiplication operation in G_1 , E the exponentiation operation in G_2 , and P the pairing operation. Other lightweight operations (e.g., hashing, symmetric encryption) are denoted by r . Let q denote the number of attributes in the access policy.

subsequent accesses, our protocol avoids all heavy operations such as scalar multiplications, pairings, or exponentiations. Instead, it relies solely on a set of very lightweight operations denoted as r , which makes our scheme extremely efficient during subsequent accesses.

7. Discussion

Our proposed access control scheme is designed to complement existing standards such as OAuth 2.0, XACML, and other fine-grained access control mechanisms, particularly in distributed IoT environments where traditional mechanisms often face scalability and efficiency challenges. While OAuth 2.0 provides a robust framework for authorization delegation, it primarily relies on centralized token issuance and validation through an authorization server, which may introduce latency and bottlenecks in highly distributed IoT networks. In contrast, our scheme leverages edge nodes as decentralized controllers that manage users credential requests for IoT objects during the first access. However, they neither authorize nor deny access to actions of IoT object, which are performed locally and independently by each IoT object. As a result, users do not need to interact with a centralized server for every access request, thus reducing dependency on a central authority and minimizing communication overhead. Furthermore, our model can be seamlessly integrated with OAuth 2.0, where the authorization server acts as an edge node that manages the actions of IoT objects and their corresponding tokens, while IoT devices represent the protected resources.

Additionally, the tokens are generated and managed using MA-ABE to enable a “one-to-many” access control paradigm. In this configuration, an encrypted token can be securely shared by the edge node with multiple users who possess the required attributes, without the need to generate new tokens for each access request. This optimized sharing capability is particularly advantageous in large-scale IoT environments, where minimizing communication costs and validation delays is crucial for system performance. Unlike conventional approaches that require frequent token regeneration or constant interactions with a central authorization server, our scheme allows for smoother and more efficient distribution of access rights. Nevertheless, each IoT device still performs local validation to ensure the authenticity and associated permissions of the received token, preserving security guarantees while optimizing resource usage.

This synergy between our decentralized control logic and existing standards like OAuth 2.0 and XACML demonstrates that our scheme not only aligns with established models but also extends their applicability to edge-based IoT deployments, offering a scalable, efficient, and secure mechanism for fine-grained access control in distributed networks.

An important consideration in real-world applications of our scheme is the synchronization of nonces (NS) between users and IoT devices. Proper synchronization is crucial to avoid desynchronization issues that could disrupt communication or cause access denial. Such issues may arise if nonce updates are not performed simultaneously by both the user and the IoT device, or if network interruptions occur during the synchronization process. In our experimental evaluations, the protocol

demonstrated robust synchronization reliability in the majority of scenarios, confirming its effectiveness under normal operating conditions. Furthermore, since our work is the first to explore task-level access control on IoT objects using MA-ABE, it lays the foundation for new research directions and further contributions in this emerging area. Regarding privacy preservation, although our current contribution does not neglect this aspect since user identities never appear in plaintext and are always hashed to prevent identity exposure. In future work, we aim to investigate privacy preservation more deeply. In particular, we plan to hide the actions requested by users, prevent tracking of user or object mobility patterns, and conceal behavioral habits that may be inferred from users or devices.

In addition, we demonstrate that our protocol optimizes storage by grouping object actions into categories at the edge server and assigning an access token to each category. In addition, IoT objects store only essential user-related information, such as nonces, thereby adhering to their strict memory constraints. This approach significantly reduces the memory footprint on resource-constrained devices. However, this process can be further enhanced by developing advanced storage optimization strategies for large-scale deployments. For instance, access request history could be leveraged to automatically remove data associated with less active users, as well as tokens nearing expiration, thereby ensuring a balance between storage efficiency and fast access control.

Finally, access-right revocation is not addressed in the current work. Future extensions of our proposal can also focus on designing scalable revocation mechanisms at both IoT devices and edge nodes. The goal in that case will be to minimize the overhead typically associated with global re-encryption or large-scale key redistribution limitations commonly observed in traditional attribute-based schemes. By enabling edge nodes and IoT devices to manage revocation tasks independently, the overall approach will further enhance the scalability, efficiency, and resilience of access control in large-scale IoT environments.

8. Conclusion

In this paper, we proposed an efficient and decentralized access control mechanism for IoT environments based on a one-to-many token strategy. Our protocol supports collaboration among multiple authorities, where each authority manages its own users based on specific attributes. Using a MA-ABE scheme, our approach enforces fine-grained access policies that allow only authorized users to perform specific actions on IoT devices. Authorization is granted according to attributes issued by cooperating authorities within the system, which enable users to decrypt tokens protected by the MA-ABE scheme. Moreover, our solution adopts Shamir’s secret sharing scheme to guarantee the uniqueness of each user’s credentials and to protect the protocol against impersonation attacks.

Furthermore, in our protocol, users can obtain credentials for different actions on IoT devices from edge nodes and reuse them for subsequent accesses without needing to request new credentials each time. Each IoT object is also able to check access rights locally, without relying on a centralized server, which reduces latency and improves scalability. Finally, we provided a formal security analysis showing that the protocol is resilient against common attack vectors, including man-in-the-middle, impersonation, and token forgery. We also demonstrated its applicability and efficiency through a smart manufacturing use case, highlighting its low computational, storage, and energy overhead. For future work, we aim to work on the synchronization of information across edge nodes to ensure real-time distribution and continuous system availability. This includes enabling edge nodes in different zones to securely share access credentials of the IoT objects under their control, while preserving the confidentiality of sensitive data. Additionally, we plan to conduct a more in-depth investigation into privacy-preserving techniques and mechanisms to further enhance the security and confidentiality of private information.

CRedit authorship contribution statement

Hicham Degdeg: Writing – review & editing, Writing – original draft, Software, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Mohamed Lehsaini:** Visualization, Validation, Supervision, Project administration. **Youcef Imine:** Validation, Supervision, Project administration.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: HICHAM DEGDEG reports administrative support, equipment, drugs, or supplies, statistical analysis, travel, and writing assistance were provided by Abou Bekr Belkaid University Tlemcen. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

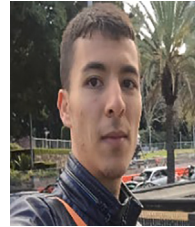
- Aghili, S.F., Sedaghat, M., Singelée, D., Gupta, M., 2022. MLS-ABAC: Efficient multi-level security attribute-based access control scheme. *Future Gener. Comput. Syst.* 131, 75–90.
- Al Sadawi, A., Hassan, M.S., Ndiaye, M., 2021. A survey on the integration of blockchain with IoT to enhance performance and eliminate challenges. *IEEE Access* 9, 54478–54497.
- Ali, G., Ahmad, N., Cao, Y., Asif, M., Cruickshank, H., Ali, Q.E., 2019. Blockchain based permission delegation and access control in Internet of Things (BACI). *Comput. Secur.* 86, 318–334.
- Ameer, S., Benson, J., Sandhu, R., 2020. The EGRBAC model for smart home IoT. In: *Proceedings of the 21st International Conference on Information Reuse and Integration for Data Science. IRI, IEEE, Las Vegas, USA*, pp. 457–462.
- Ameer, S., Benson, J., Sandhu, R., 2022. An attribute-based approach toward a secured smart-home IoT access control and a comparison with a role-based approach. *Information* 13 (2), 60.
- Ameer, S., Sandhu, R., Benson, J., 2023. Hybrid approaches (ABAC and RBAC) toward secure access control in smart home IoT. *IEEE Trans. Dependable Secur. Comput.* 20 (5), 4032–4051.
- Bauer, M., Bui, N., De Loof, J., Magerkurth, C., Nettsträter, A., Stefa, J., Walewski, J.W., 2013. IoT reference model. In: *Enabling Things To Talk: Designing IoT Solutions with the IoT Architectural Reference Model. Springer Berlin Heidelberg Berlin, Heidelberg*, pp. 113–162.
- Beltrán, M., 2018. Identifying, authenticating and authorizing smart objects and end users to cloud services in Internet of Things. *Comput. Secur.* 77, 595–611.
- Das, S., Namasudra, S., 2022. Multiauthority CP-ABE-based access control model for IoT-enabled healthcare infrastructure. *IEEE Trans. Ind. Informatics* 19 (1), 821–829.
- Dunkels, A., Gronvall, B., Voigt, T., 2004. Contiki-a lightweight and flexible operating system for tiny networked sensors. In: *Proceedings of the 29th Annual International Conference on Local Computer Networks. IEEE, Tampa, USA*, pp. 455–462.
- Fedrecheski, G., De Biase, L.C.C., Calcina-Ccori, P.C., Zuffo, M.K., 2018. Attribute-based access control for the swarm with distributed policy management. *IEEE Trans. Consum. Electron.* 65 (1), 90–98.
- Gubbi, J., Buyya, R., Marusic, S., Palaniswami, M., 2013. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Gener. Comput. Syst.* 29 (7), 1645–1660.
- Hao, X., Ren, W., Fei, Y., Zhu, T., Choo, K.-K.R., 2022. A blockchain-based cross-domain and autonomous access control scheme for Internet of Things. *IEEE Trans. Serv. Comput.* 16 (2), 773–786.
- Huang, W., Xie, X., Wang, Z., Feng, J., Han, G., Zhang, W., 2023. ZT-access: A combining zero trust access control with attribute-based encryption scheme against compromised devices in power IoT environments. *Ad Hoc Networks* 145, 103161.
- Khalid, M., Hameed, S., Qadir, A., Shah, S.A., Draheim, D., 2023. Towards SDN-based smart contract solution for IoT access control. *Comput. Commun.* 198, 1–31.
- Kusiak, A., 2018. Smart manufacturing. *Int. J. Prod. Res.* 56 (1–2), 508–517.
- Lewko, A., Waters, B., 2011. Decentralizing attribute-based encryption. In: *Proceedings of the 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques. Springer, Tallinn, Estonia*, pp. 568–588.
- Li, F., Han, Y., Jin, C., 2016. Practical access control for sensor networks in the context of the Internet of Things. *Comput. Commun.* 89, 154–164.
- Li, X., Liu, T., Chen, C., Cheng, Q., Zhang, X., Kumar, N., 2022. A lightweight and verifiable access control scheme with constant size ciphertext in edge-computing-assisted IoT. *IEEE Internet Things J.* 9 (19), 19227–19237.
- Li, D., Liu, J., Wu, Q., Guan, Z., 2019. Efficient CCA2 secure flexible and publicly-verifiable fine-grained access control in fog computing. *IEEE Access* 7, 11688–11697.
- Liu, H., Han, D., Li, D., 2020. Fabric-IoT: A blockchain-based access control system in IoT. *IEEE Access* 8, 18207–18218.
- Makhdoom, I., Zhou, I., Abolhasan, M., Lipman, J., Ni, W., 2020. PrivySharing: A blockchain-based framework for privacy-preserving and secure data sharing in smart cities. *Comput. Secur.* 88, 101653.
- Mandal, S., Bera, B., Sutrala, A.K., Das, A.K., Choo, K.-K.R., Park, Y., 2020. Certificateless-signcryption-based three-factor user access control scheme for IoT environment. *IEEE Internet Things J.* 7 (4), 3184–3197.
- Oh, S.-R., Kim, Y.-G., Cho, S., 2019. An interoperable access control framework for diverse IoT platforms based on oauth and role. *Sensors* 19 (8), 1884.
- Osterlind, F., Dunkels, A., Eriksson, J., Finne, N., Voigt, T., 2006. Cross-Level Sensor Network Simulation with COOJA. In: *Proceedings of the 31st International Conference on Local Computer Networks. IEEE, Tampa, USA*, pp. 641–648.
- Ouaddah, A., Mousannif, H., Abou Elkalam, A., Ait Ouahman, A., 2017. Access control in the Internet of Things: Big challenges and new opportunities. *Comput. Netw.* 112, 237–262.
- Pal, S., Hitchens, M., Varadharajan, V., Rabehaja, T., 2019. Policy-based access control for constrained healthcare resources in the context of the Internet of Things. *J. Netw. Comput. Appl.* 139, 57–74.
- Ravidas, S., Conti, M., Lal, C., 2019. Access control in Internet of Things: a survey. *J. Netw. Comput. Appl.* 144, 79–101.
- Sarma, R., Kumar, C., Barbhuiya, F.A., 2022. MACFI: A multi-authority access control scheme with efficient ciphertext and secret key size for fog-enhanced IoT. *J. Syst. Archit.* 123, 102347.
- Selander, G., Mattsson, J., Palombini, F., Seitz, L., 2019. Object Security for Constrained Restful environments (OSCORE). *Tech. Rep., Internet Engineering Task Force*.
- Shamir, A., 1979. How to share a secret. *Commun. ACM* 22 (11), 612–613.
- Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. *IEEE Internet Things J.* 3 (5), 637–646.
- Sivaselvan, N., Bhat, K.V., Rajarajan, M., Das, A.K., 2023. A new scalable and secure access control scheme using blockchain technology for IoT. *IEEE Trans. Netw. Serv. Manag.* 20 (3), 2957–2974.
- Stouffer, K., Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., et al., 2023. Guide to Operational Technology (OT) Security. US Department of Commerce, National Institute of Standards and Technology ...
- Sun, S., Du, R., Chen, S., Li, W., 2021. Blockchain-based IoT access control system: towards security, lightweight, and cross-domain. *IEEE Access* 9, 36868–36878.
- Sun, P., Wan, Y., Wu, Z., Fang, Z., Li, Q., 2025. A survey on privacy and security issues in IoT-based environments: Technologies, protection measures and future directions. *Comput. Secur.* 148, 104097.
- Sundmaeker, H., Guillemin, P., Friess, P., Woelfflé, S., et al., 2010. Vision and challenges for realising the Internet of Things. *Clust. Eur. Res. Proj. the Internet Things, Eur. Commision* 3 (3), 34–36.
- Trabelsi, R., Fersi, G., Jmaiel, M., 2023. Access control in Internet of Things: A survey. *Comput. Secur.* 135, 103472.
- Wang, W., Huang, H., Yin, Z., Gadekallu, T.R., Alazab, M., Su, C., 2023. Smart contract token-based privacy-preserving access control system for industrial Internet of Things. *Digit. Commun. Networks* 9 (2), 337–346.
- Xiang, X., Cao, J., Fan, W., 2022. Decentralized authentication and access control protocol for blockchain-based e-health systems. *J. Netw. Comput. Appl.* 207, 103512.
- Yang, H., Kumara, S., Bukkapatnam, S.T., Tsung, F., 2019. The Internet of Things for smart manufacturing: A review. *IIEE Trans.* 51 (11), 1190–1216.
- Yao, X., Zhou, J., Du, X., Zhang, S., 2024. A CP-ABE and IOTA based lightweight sensitive data access control scheme for IoT. *IEEE Internet Things J.*
- Zaidi, T., Usman, M., Aftab, M.U., Aljuaid, H., Ghadi, Y.Y., 2023. Fabrication of flexible role-based access control based on blockchain for Internet of Things use cases. *IEEE Access* 11, 106315–106333.
- Zhang, Y., Chen, L., Zhu, Y., Kong, X., 2025. Privacy-enhanced role-based access control for IoT systems. *IEEE Internet Things J.* 1–1.
- Zhang, Y., Nakanishi, R., Sasabe, M., Kasahara, S., 2021. Combining IOTA and attribute-based encryption for access control in the Internet of Things. *Sensors* 21 (15), 5053.
- Zhang, L., Zhang, Y., Wu, Q., Mu, Y., Rezaeibagha, F., 2022. A secure and efficient decentralized access control scheme based on blockchain for vehicular social networks. *IEEE Internet Things J.* 9 (18), 17938–17952.
- Zheng, P., Wang, H., Sang, Z., Zhong, R.Y., Liu, Y., Liu, C., Mubarak, K., Yu, S., Xu, X., 2018. Smart manufacturing systems for Industry 4.0: Conceptual framework, scenarios, and future perspectives. *Front. Mech. Eng.* 13, 137–150.



DEGDEG Hicham is currently a Ph.D. student at the University of Abou Bekr Belkaid Tlemcen. His research interests include secure data sharing, privacy-preserving techniques, cryptographic access control, cybersecurity in emerging technologies such as Fog computing and the Internet of Things (IoT), and communication reliability in heterogeneous IoT networks. He received his Master's degree in Distributed Networks and Systems in 2019, where he focused on the implementation of blockchain-based access control systems using Ethereum smart contracts. In addition to his academic work, he is the CEO and founder of DGSOFTWARE, a company specializing in digital systems development.



Mohamed LEHSAINI received his Ph.D. (2009) from the University of Franche-Comté, France and the University of Tlemcen, Algeria. In June 2013, he was awarded the qualification for supervising research (HDR - Habilitation à Diriger des Recherches in french) from Sidi Bel-Abbes University, Algeria. Since 2000, he is a professor at the University of Tlemcen, Algeria, and since 2015, he is the Head of the research STIC Laboratory (Information and Communication Technology Systems) located at the University of Tlemcen. His current research interests include Vehicular Ad hoc



Networks, Ad hoc and Sensor Networks, Internet of Things, Internet of Multimedia Things, Wireless and Mesh Networks, Distributed Systems, and Security in networks. He is the author/co-author of several research papers in national and international conferences and journals. He is member in several research projects, reviewer in several international conferences and journals.

Youcef Imine received the master's degree in networking and distributed systems from Abou Bekr Belkaid University (Algeria), in 2016 and the PhD degree in computer science from the University of Technology of Compiègne (France), in 2019. He is an associate professor with INSA Hauts-de-France (Université Polytechnique de Hauts-de-France). His main works concern communication management and data security in emerging technologies such as Fog computing and IoT, which include several challenges such as data authentication, cryptographic access control, privacy-preserving and trust management and communication reliability in heterogeneous IoT networks.